# BIOARC: Discovering Optimal Neural Architectures for Biological Foundation Models

**Yi Fang** [1] **Haoran Xu** [1] **Jiaxin Han** [2] **Sirui Ding** [3] **Yizhi Wang** [4] **Yue Wang** [4] **Xuan Wang** [† 1]

## Abstract

Foundation models have revolutionized AI, yet biological applications often repurpose general architectures without accounting for the intrinsic structural and functional properties of distinct modalities, such as genomic and proteomic sequences. Consequently, these architectures lack the inductive biases required to capture the complex "grammars" inherent to biological data, resulting in suboptimal performance. To address this, we introduce BIOARC, a framework utilizing Neural Architecture Search (NAS) to shift from intuition-driven design to automated data-driven discovery. Unlike standard NAS restricted to homogeneous spaces, BIOARC navigates a heterogeneous space for open-ended composition of architectural blocks. By systematically analyzing the interplay between architecture, tokenization, and training across modalities, BIOARC identifies novel hybrid architectures that surpass state-of-the-art models while being up to 25x smaller. We distill these findings into empirical design principles and validate their biological relevance, demonstrating how our designs hierarchically capture the underlying biological grammar. Additionally, we introduce an agentic framework to predict optimal architectures for new tasks. Overall, BIOARC provides a data-driven methodology for developing the next generation of efficient biological foundation models and task-specific networks.

[1]Department of Computer Science, Virginia Tech, Blacksburg, VA, USA [2]Department of Computer Science, Carnegie Mellon University, Pittsburgh, PA, USA [3]Department of Biomedical Data Science, Stanford University, Stanford, CA, USA [4]Department of Electrical and Computer Engineering, Virginia Tech, Blacksburg, VA, USA. Correspondence to: Xuan Wang <xuanw@vt.edu>.

*Proceedings of the $43^{rd}$ International Conference on Machine Learning*, Seoul, South Korea. PMLR 306, 2026. 

## 1. Introduction

The advent of foundation models, large-scale neural networks pretrained on vast amounts of data, has catalyzed a profound revolution in artificial intelligence (AI). In general domains such as natural language processing (NLP) (Devlin et al., 2019; OpenAI, 2023; Touvron et al., 2023) and computer vision (CV) (Dosovitskiy, 2020; Liu et al., 2024), foundation models built upon architectures such as Transformers (Vaswani et al., 2017) and Diffusion Models (Ho et al., 2020) have demonstrated unprecedented capabilities, reshaping research and applications. This success has spurred a wave of interest in applying this paradigm to biology, a trend driven by the increasing availability of massive datasets. The promise is to leverage large-scale data to create powerful biological foundation models (Xiao et al., 2025) that can learn the underlying grammar of genetic or protein sequences, accelerating discovery in drug development, synthetic biology, and personalized medicine.

However, directly applying architectures from general AI domains presents numerous limitations in biology. Most current biological foundation models (Zhou et al., 2024; Rives et al., 2021; Xiao et al., 2025) are built upon Transformers (Vaswani et al., 2017), which was originally designed for human language and not for the complex "grammar" unique to biological data, making it suboptimal for biological applications. Specifically, biological sequences present a dual challenge: they require processing extremely long contexts (e.g., whole genomes) (Wang et al., 2017) that incur prohibitive computational costs for standard Transformers, while simultaneously demanding the capture of precise local structural motifs (Greener et al., 2022), which is an inductive bias not inherently prioritized by global attention mechanisms. The limitations of repurposing general architectures for biological sequences necessitate a fundamental inquiry: what constitutes an optimal neural architecture for biological foundation models?

Answering this question is non-trivial due to three distinct technical challenges. **First,** architectural innovation in this domain is hampered by a lack of established guiding principles. This presents a stark contrast to a field like NLP, where language is a human-generated system with rules and structures we inherently understand. Biological in-

formation, on the other hand, is designed by nature. Its underlying principles, such as the physicochemical laws governing protein design (Marcos et al., 2017; Dill & MacCallum, 2012), are yet to be fully discovered and remain largely unknown. Consequently, unlike the NLP field, there is no universally acknowledged architecture that consistently excels across the board, forcing a reliance on manual, intuition-driven design that is inefficient (Sapoval et al., 2022; Eisenstein, 2024; Vishniakov et al., 2025). **Second**, while automated search methods like Neural Architecture Search (NAS), standard methods typically focuses on *micro-level* optimizations within homogeneous spaces, such as tuning layer counts or filter sizes in purely convolutional networks. Even domain-specific hybrid efforts, such as GenomeNet-Architect (Gündüz et al., 2024), primarily rely on parameterizing fixed design templates distilled from existing models such as DeepVirFinder (Ren et al., 2020), rather than exploring the open-ended combination of computational paradigms. Without a strategy to construct representative search spaces, such methods are confined to optimizing known backbones, failing to uncover novel topologies within the heterogeneous landscape. **Finally,** adding to the complexity is the deep entanglement of architecture, data preprocessing, and training strategy. The efficacy of a given architecture is highly sensitive to choices in tokenization and optimization schemes, implying that architectural choices cannot be effectively designed, developed, and evaluated in isolation.

To address these challenges, we introduce **BIOARC**, a framework that shifts the paradigm from inefficient manual design to automated data-driven discovery. By navigating a heterogeneous search space composed of five distinct operation primitives, **BIOARC** enables the open-ended composition of complementary inductive biases. **Our contributions are as followed: First**, we discover novel hybrid architectures that achieve state-of-the-art performance across diverse biological tasks while being up to $25\times$ smaller than existing foundation models. **Second**, serving as a unified testbed, **BIOARC** disentangles the complex interplay between architecture, tokenization, and optimization, providing actionable design guidelines for future biological modeling. **In addition**, through fine-grained interpretability analysis, we demonstrate that the discovered topologies hierarchically capture known biological regulatory mechanisms. **Finally**, bridging analysis and automation, we introduce an agentic framework capable of efficiently identifying optimal architectures for unseen tasks without exhaustive search.

## 2. Related Work

**Foundation Models for Biological Data** Inspired by NLP and CV, biological foundation models learn patterns from vast unlabeled sequences via self-supervised pretraining. In proteomics, transformer-based models like AlphaFold (Jumper et al., 2021) revolutionized structure prediction. Scaling to the whole-genome level, Evo-1 and Evo-2 (Nguyen et al., 2024; Brixi et al., 2025) enables generative design. For sequence representation, models such as PathoLM (Dip et al., 2024), Nucleotide Transformer (Dalla-Torre et al., 2023), Gena-LM (Fishman et al., 2023), and VQDNA (Li et al., 2024) target DNA; ProtBert (Elnaggar et al., 2021) and ESM-1 and ESM-2 (Rives et al., 2021; Lin et al., 2023) focus on proteins; while RNAErnie (Wang et al., 2024) and RNABERT (Akiyama & Sakakibara, 2022) address RNA. Expanding modalities, Geneformer (Theodoris et al., 2023) and CellFM (Zeng et al., 2025) targets single-cell gene expression value sequence.

**Hybrid Architecture** Beyond pure sequence models, hybrid architectures combine distinct neural mechanisms to leverage complementary strengths. In protein science, integrating GNNs with attention captures both sequential context and geometric constraints (Jumper et al., 2021). Similarly, for nucleic acids, merging CNNs with Transformers enables efficient processing of high-resolution sequences; CNNs encode local elements and reduce length, allowing Transformers to model distal interactions such as enhancer-promoter contacts (Avsec et al., 2021; Yu et al., 2024).

**Neural Architecture Search** Neural Architecture Search (NAS) automates architecture design. While early RL (Zoph & Le, 2017) and evolutionary (Real et al., 2017) methods were computationally expensive, efficient one-shot techniques (Liu et al., 2019) have since enabled landmarks (Tan & Le, 2020; Deng et al., 2009).BossNAS (Li et al., 2021) demonstrate a hybrid CNN-Transformer network but limited two-way combinations. However, biological applications remain limited. Prior works focus mostly on single architectures (Zhang et al., 2021a;b) or specific sequence tasks (Sivangi et al., 2022). Even GenomeNet-Architect (Gündüz et al., 2024), which combines CNNs and RNNs, is still constrained by 2 module types.

**Architecture Prediction** To mitigate the high computational cost of NAS, architecture predictors serve as efficient surrogates, estimating performance directly from structural properties (Ying et al., 2019). Trained on pre-evaluated architecture-performance pairs, these models range from simple regressors to Graph Neural Networks capable of encoding complex topologies (Łukasz Dudziak et al., 2021). However, their generalization is inherently limited by the scope of training tasks (Duan et al., 2021), restricting transferability to novel domains without significant retraining.

## 3. BIOARC Framework

This section introduces BIOARC, a framework for discovering optimal heterogeneous backbones. First we introduce how to define the search space and how to construct the supernet in Section 3.1. Then we elaborate on the training of supernet in Section 3.2. Last we discuss how to evaluation and rank all architectures in Section 3.3. While demonstrated on DNA and protein, the framework generalizes to other modalities such as RNA and single-cell data.

### 3.1. Search Space & Supernet

As discussed, previous NAS methods have focused on micro-level optimizations within homogeneous spaces or on no more than 2-way hybrid architectures. To capture different inductive biases in biological sequence, a heterogeneous ensemble of multiple architecture blocks is needed.

**Basic Blocks** We begin by introducing the basic blocks defined as $\mathcal{M}$, including a variety of architectures widely used for biological data such as **CNN** (Krizhevsky et al., 2012), **LSTM** (Hochreiter & Schmidhuber, 1997), **Transformer** (Vaswani et al., 2017), **Mamba** (Gu & Dao, 2024) and **Hyena** (Poli et al., 2023). The diversity in block types ensures our search can cover different inductive biases. Details of these blocks can be found in Appendix A.2.

**Paths formed by blocks** Each path $a \in \mathcal{A}$ is a sequence of basic blocks $(l_1, l_2, \ldots, l_d)$, where its structure is determined by three variations:

1. ***Network Depth*** $(\mathbf{d})$: The number of blocks in a path, chosen from a predefined set of possible depths, $\mathcal{D} = \{D_{\min}, \ldots, D_{\max}\}$.
2. ***Block Type*** $(\mathbf{m})$: A tuple $\mathbf{m} = (m_1, m_2, \ldots, m_d)$ of length $d$, where each element $m_i \in \mathcal{M}$ specifies the block for the $i$-th layer.
3. ***Hidden Dimension*** $(\mathbf{h})$: A tuple $\mathbf{h} = (h_1, h_2, \ldots, h_d)$ of length $d$, where each $h_i$ is a hidden dimension selected from a set of possible widths $\mathcal{H}$.

A complete path $a$ is constructed from a tuple $(\mathbf{h}, \mathbf{m})$ of length $d$. For $i \in \{1, \ldots, d\}$, the $i$-th layer $l_i$ is of type $m_i$ and maps an input of dimension $h_{i-1}$ to an output of dimension $h_i$. The initial dimension $h_0$ is the embedding dimension which is a fixed hyperparameter.

**Search Space Design** Let $\mathcal{C}^{(d)}_{\text{dim}}$ and $\mathcal{C}^{(d)}_{\text{type}}$ be the sets of all permissible dimension and block-type configurations for a given depth $d$, respectively. The total search space $\mathcal{A}$ is then the union of the Cartesian products of these configuration sets across all possible depths:

$$\mathcal{A} = \bigcup_{d \in \mathcal{D}} \left( \mathcal{C}^{(d)}_{\text{dim}} \times \mathcal{C}^{(d)}_{\text{type}} \right) \tag{1}$$

The combinatorial nature of the configuration sets yields millions of candidates. In standard homogeneous search spaces, where variations are purely parametric (e.g., filter sizes), uniform random sampling is often sufficient to cover the manifold. However, in our heterogeneous space, distinct topological families (e.g., pure CNNs vs. Hyena-Transformer hybrids) are unevenly distributed. Naive sampling would likely overfit to structurally redundant patterns while neglecting rare but potent hybrid combinations. To ensure computational tractability while preserving structural diversity, we employ a Representative Sampling Strategy to extract a concise subset of 360 architectures. This process systematically prunes topologically redundant paths via **distance-based filtering**, utilizing a metric based on log-transformed dimensions. Furthermore, we incorporate **a monotonic width constraint with the maximum dimension fixed at the last layer**. This intentionally engineers a structural bias where wider, parameter-heavy blocks appear in a greater number of valid paths. Consequently, this design ensures parameter-heavy blocks implicitly receive higher sampling frequency during training, aligning training intensity with block complexity to facilitate convergence. Finally, we use **K-Means clustering on the one-hot encoded architecture vectors** to control the number of paths in the search space, selecting the centroids of these clusters. Details are provided in Appendix A.3.

**Supernet Construction** We employ a weight-sharing supernet approach(Bender et al., 2018). The key insight is that many architectures within $\mathcal{A}$ reuse identical building blocks. For example, a specific Transformer block with a 256 input dimension and a 512 output dimension is a component in different paths. Therefore, the supernet is a single large network that effectively contains all candidate architectures as paths. First, we identify the set of all unique blocks used in paths across the entire search space:

$$L = \{l \mid \exists a \in \mathcal{A}, l \in a\} \tag{2}$$

Each unique block is defined by its type (e.g., CNN) and its dimensions. Other hyperparameters (e.g., CNN kernel sizes, Transformer attention heads) are fixed. The supernet weight set $W$ is the union of all unique block weights:

$$W = \bigcup_{l \in L} W_l \tag{3}$$

where $W_l$ are the trainable parameters for a unique layer $l$. The weights for any specific path denoted as $w(a)$ are a subset of these shared weights:

$$w(a) = \{W_{l_i} \mid l_i \in a\} \subseteq W \tag{4}$$

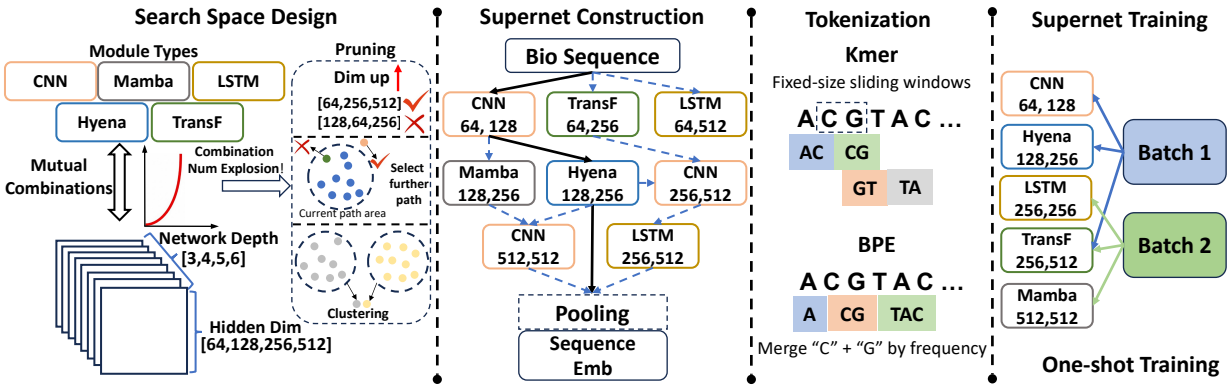



*Figure 1.* This figure provides an overview of the four core stages of the BioArc framework. (1) Search Space Design: Defines diverse block types, network depths, and hidden dimensions, using pruning and clustering strategies to manage the combinatorial explosion. (2) Supernet Construction: Encodes the vast search space into a single, weight-sharing Supernet, with each path representing a candidate architecture. (3) Supernet Training: Adopts a one-shot methodology, sampling and optimizing different paths across batches for efficiency.

### 3.2. Supernet Pretraining

**One-Shot Supernet Pretraining** We adopt a random path sampling mechanism inspired by the Single Path One-Shot approach (Guo et al., 2020). In each forward pass, we activate a single path through the supernet. We sample paths from a uniform distribution. Leveraging the structural bias engineered into our search space, this simple sampling strategy naturally fulfills the higher data requirements of complex blocks. Consequently, the stochasticity of single-path updates acts strictly as a regularization mechanism. By decoupling specific block interactions, it prevents co-adaptation (Lan et al., 2020), ensuring that shared blocks learn robust, independent features rather than overspecializing to specific neighbors.

**Self-Supervised Pretraining Objectives** The supernet denoted $\mathcal{A}$ is trained by stochastically sampling architectures from the search space. In each training step, a path $a$ is sampled, and the supernet's shared weights $W$ are updated by minimizing self-supervised loss $\mathcal{L}$. The overall training objective is to optimize the expectation of this loss over the distribution of all possible architectures:

$$\min_{W} \mathbb{E}_{a\sim\mathcal{A}}[\mathcal{L}(\mathcal{A}(X; w(a)))] \tag{5}$$

where $X$ is the input data, $a \sim \mathcal{A}$ denotes a path uniformly sampled from the search space $\mathcal{A}$ defined in Section 3.1, and $w(a) \subset W$ are the weights corresponding to the block in path $a$. This ensures every operation in the search space is trained equally to provide a stable performance proxy.

To ensure the supernet learns meaningful and generalizable features and to investigate the impact of different pretraining tasks, the supernet is pretrained using one of three distinct self-supervised learning objectives: Masked Modeling (MM), Contrastive Learning (CL), and Next Token Prediction (NTP). In Equation 5, $\mathcal{L}$ refers to the specific loss function corresponding to the chosen pretraining strategy. This pretraining step is performed on a large corpus of unlabeled biological data before the supernet is used for evaluation on specific downstream tasks. The detailed mathematical formulations and biological motivations for MM, CL, and NTP are provided in Appendix A.4.

### 3.3. Architecture Evaluation and Ranking

**Evaluation Protocol** To fairly assess the performance of sampled architectures, **we optimize each path independently, rather than fine-tuning the entire supernet as a whole**. We adopt this approach for two primary reasons. First, the computational cost is tractable due to the significantly smaller size of downstream datasets compared to the pretraining corpus. Second, decoupling the weights is essential to eliminate interference, thereby ensuring **optimal performance and accurate ranking**. This isolation is particularly crucial because the supernet is trained to prevent **co-adaptation** (Bender et al., 2018; Guo et al., 2020), resulting in shared parameters that act as a **compromise** for expected performance rather than being optimal for any specific path. We conduct this independent assessment using two paradigms distinct by their weight initialization:

**Finetuning** The parameters of the sampled architecture $a$, denoted as $w(a)$, are initialized by inheriting the corresponding weights from the pretrained supernet $W$. The architecture is finetuned to minimize the task-specific loss:

$$\min_{w(a)} \mathcal{L}_{\text{task}}(\mathcal{A}(X_{\text{task}}; w(a))), \text{ s.t. } w(a)_{\text{init}} \subset W. \tag{6}$$

**Architecture Ranking Strategy** We rank the sampled architectures $\mathbb{S}$ based on their aggregated performance across the set of downstream tasks $\mathcal{T}$. Since tasks utilize different metrics with varying scales and statistical variances (e.g.,

Accuracy vs. RMSE), a direct summation would disproportionately favor tasks with wider numerical distributions. To address this heterogeneity and ensure that each task contributes equally to the final ranking, we employ Z-score normalization to standardize the performance metrics before aggregation. We define the unified score for an architecture $a$ as the mean of these standardized metrics:

$$\text{Score}(a) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} s_t \cdot \frac{P_t(a) - \mu_t}{\sigma_t}, \tag{7}$$

where $P_t(a)$ denotes the raw performance of architecture $a$ on task $t$. The terms $\mu_t$ and $\sigma_t$ represent the mean and standard deviation of the performance scores for task $t$ across all sampled candidates. The coefficient $s_t \in \{1, -1\}$ aligns the optimization direction, taking 1 for metrics where higher is better (e.g., Accuracy) and $-1$ for error-based metrics (e.g., RMSE). Finally, the top candidates $\mathbb{A}_{\text{top}}$ are identified by selecting the $k$ architectures with the highest Score$(a)$.

**Optimal Architecture as Foundation Model** Once the top ranking architecture $a^* \in \mathcal{A}$ is identified, we randomly initialize its parameters, denoted as $\theta$, and pretrain it:

$$\min_{\theta} \mathcal{L}(a^*(X; \theta)). \tag{8}$$

To evaluate the foundation model's transferability, we subsequently finetune the pretrained parameters $\theta$ on each specific downstream task $t$:

$$\min_{w} \mathcal{L}_{\text{task}}(\mathcal{A}(X_{\text{task}}; w)) \text{ s.t. } w_{\text{init}} \leftarrow \theta \tag{9}$$

## 4. Experiment

Having detailed our construction of BIOARC framework, we aim to answer the following research questions with our experiments. **RQ1:** Can finetuned architectures from BIOARC pretrained supernet outperform baseline methods? **RQ2:** What are the common architectural properties that characterize high-performing models? **RQ3:** Does the optimal architecture identified by BIOARC serve as a effective backbone for biological foundation models? **RQ4:** How does the interplay between architecture, tokenization, and pretraining objectives impact downstream performance? **RQ5:** Do the discovered hybrid architectures hierarchically capture the underlying biological grammar and regulatory mechanisms?

### 4.1. Settings and baselines

**DNA** For pretraining, we use the full human reference genome **GRCh38**. For downstream evaluation, we employ the human subset of the **GUE benchmark** (Zhou et al., 2024), comprising 12 datasets to cover a variety of tasks. Detailed descriptions are provided in Appendix A.1.1.

**Protein** For pretraining, we use a 10% randomly sampled **UniRef50** (Suzek et al., 2007) subset, containing 72.1M representative sequences clustered at 50% identity. For evaluation, we select 6 tasks from the **PEER** benchmark (Xu et al., 2022), covering protein function, structure, and interactions. Detailed descriptions are provided in Appendix A.1.2.

**Baselines** For DNA, we use Nucleotide Transformer (Dalla-Torre et al., 2023), DNABERT-2 (Zhou et al., 2024), and VQDNA (Li et al., 2024) as baselines. These pretrained large foundation models all share human DNA GRCh38 in pretraining with finetuing on additional multi-species genome datasets. For protein, we use ESM-1 and ESM-2 (Rives et al., 2021; Lin et al., 2023) and ProtBert (Elnaggar et al., 2021) as baselines. ProtBert is pretrained on BFD (Steinegger et al., 2019) with 393B amino acids and ESM-1b is pretrained on UniParc (Consortium, 2007) with 86B amino acids. More details of training setting could be found in Appendix A.5.1.

**Tokenization** To thoroughly investigate the impact of tokenization on performance, we conducted experiments using different tokenizers, including k-mer and Byte Pair Encoding (BPE) (Sennrich et al., 2016). ***K-mer tokenizers*** operate by dividing a sequence into overlapping or non-overlapping substrings of a fixed length, denoted as $k$. ***BPE*** is a subword tokenization strategy that begins with a vocabulary of individual characters and iteratively merges the most frequently occurring adjacent pairs of tokens into a new, single token.

### 4.2. Main Results

In this section, we address RQ1, RQ2 and RQ3. **For RQ1**, we observe the following. **First, architectures discovered by BIOARC are highly competitive, often outperforming larger, pretrained models with much smaller model size.** As detailed in Table 1, BIOARC models **achieve superior performance across all DNA tasks**, in some cases by a substantial margin. **For the protein tasks** in Table 2, we disentangle architecture from pretraining budget through a controlled comparison in which BIOARC 8M and a reimplemented ESM-2 8M are pretrained under *identical* conditions (full UniRef50, 50K steps), making architecture the only variable. **Under this matched setting, BIOARC 8M outperforms ESM-2 8M on all six PEER tasks, including the structural Fold task (20.75 vs. 18.25).** This indicates that the apparent weakness of compact models on structural tasks is *not* an architectural ceiling but a consequence of pretraining scale: the residual gap to the released ESM-2 8M (Fold 22.14) tracks its $\sim$10$\times$ larger pretraining budget (full UniRef50, 500K steps, 32$\times$A100 vs. 50K steps, 4$\times$A100). We attribute BIOARC's advantage to task-specific inductive bias: its hybrid topology adapts to the intrinsic signal density of biological sequences, maximizing data efficiency under a

fixed budget, as further evidenced by its consistently lower pretraining loss at every checkpoint (Appendix A.6.5). **The difference in performance highlights a key principle for biology model design that optimal architecture depends on the nature of the task.** For those demanding intricate feature extraction from the sequence itself, a specialized architecture is key. For others that rely on vast, implicit priors learned from data at scale, massive pretraining is essential.

**For RQ2**, we observe three key findings. **First, top-performing architectures converge on specific structural patterns.** For DNA tasks, as shown in Figure 2, optimal models typically initiate with Hyena block to first captures long-range dependencies, with Transformer blocks in the middle to model complex contextual relationships and CNN blocks at the end to extract critical features. **Second, while optimal architectures are task-specific, they exhibit significant commonalities within task families.** For instance, the top 10% architectures for Transcription Factor Prediction-3 and 4 share 98.0% similarity as shown in Figure 8 and Appendix A.6.2.

To answer RQ3, we selected a overall top-performing BIOARC architecture and conduct pretraining similar as DNABERT2. As shown in Figure 3, **with only 1/20 model size and 1/10 training steps, BIOARC-based foundation model outperforms the human-designed architectures on downstream DNA tasks.** To further evaluate its scalability, we extended model depth to 10 layers by doubling the number of Hyena and Transformer layers and fixed hidden dimension to 1024 across all layers to expand model width. As the results shown in Appendix A.6.3, **scaling up model size further improves the performance.** BIOARC-F is also competitive with recent genomic foundation models (Caduceus (Schiff et al., 2024) and GENERator (Wu et al., 2025)) on the Nucleotide Transformer benchmark and the Genomic Benchmarks (Grešová et al., 2023), achieving the best result on all histone-mark and enhancer tasks despite being substantially smaller (Appendix A.6.4). This result, combined with our findings for RQ1, indicates that BIOARC is highly effective for identifying both high-performing specialized architectures and robust backbones for foundational models in biology. Detailed training settings could be found in Appendix A.5.1 and analysis of the computational costs is provided in Appendix A.5.3.

### 4.3. Study on Interplay of Training and Tokenization

In this section, **we address RQ4** by analyzing how critical design choices including training strategy and tokenization interact with architectural topology.

**Impact of Training Strategies** We summarize the training strategy results in Figure 5. **First, pretraining does not guarantee gains.** Training from scratch (only-ft), where we train each architecture path from random initialization on the downstream task, achieves the highest Win Rate on the PD-tata task (36.0%) and notably outperforms contrastive pretraining in other scenarios. **Secondly, no pretraining strategy dominates.** While masked modeling (mask-ft) generally yields the highest performance across TFP datasets, Next Token Prediction (ntp-ft) proves superior on the CPD-notata task. This suggests that the optimal training strategy is highly dependent on the downstream data characteristics. **Thirdly, contrastive learning exhibits instability and high variance.** In addition to generally lower Win Rates, *con-ft* demonstrates significant performance fluctuations compared to reconstruction-based methods (*mask-ft* and *ntp-ft*). This instability suggests that contrastive objectives may require longer training schedule to converge effectively, whereas generative signals offer better sample efficiency within limited training budgets.

**Impact of Tokenization** **First, we find that the optimal tokenizer choice is highly architecture-dependent.** As shown in Figure 6, the Transformer-based architecture performs best with a 6-mer tokenizer, while the CNN-based model achieves its highest scores with a 1-mer tokenizer. This reveals a deep interplay between architecture and tokenization, demanding their co-optimization. **Second, the tokenizer's effectiveness interacts with the training strategy.** As shown in Figure 3, BIOARC-F with BPE tokenizer outperforms 1-mer tokenizer, which contradicts the situation in task-specific models. This suggests that pretraining propels complex tokenization, while simple tokenization methods are less reliant on pretraining. More comprehensive results are available in the Appendix A.6.7.

### 4.4. Interpretability Case Study on Core Promoter

Visualizing layer-specific activations on the Core Promoter task (Figure 4) reveals that our model spontaneously learns a hierarchy mirroring biological complexity. Specifically, the **Global Context (Hyena, L0)** establishes broad genomic context via continuous receptive fields, forming a semantic foundation. Subsequently, **Spatial Syntax (Transformer, L1-2)** layers anchor attention on the Transcription Start Site (TSS), acting as syntactic validators. Crucially, at the **Motif Synergy (CNN, L3-5)** level, filters simultaneously activating on Inr and DPE elements confirm the unsupervised rediscovery of the **Inr+DPE synergy rule** (Burke & Kadonaga, 1996). These findings suggest a correspondence between our model's learned hierarchy and established biological principles, highlighting the interpretability of the hybrid design. Details can be found in Appendix A.8.

### 4.5. More Analysis

**Comparison with Pure Architectures** Hybrid architectures consistently outperform single-module baselines, validating the effectiveness of our search space design. Details can be

*Table 1.* Performance on different DNA tasks. **Bold** and underline indicate the best and second-best result respectively. We report the average size of top-performing architectures across tasks. More results on Protein can be found in Appendix A.6.5.

| Method | #Param | #Data | TFP | | | | | PD | | | CPD | | | SSP |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | 0 | 1 | 2 | 3 | 4 | all | notata | tata | all | notata | tata | Reconstruction |
| HyenaDNA (one-hot) | 6.6M | 3.1B | 62.30 | 67.86 | 46.85 | 41.78 | 61.23 | 47.38 | 52.24 | 5.34 | 36.95 | 35.38 | 72.87 | 72.67 |
| NT-2500M-1000g (6-mer) | 2500M | 20.5T | 66.31 | 68.30 | 58.70 | 49.08 | 67.59 | 90.95 | 93.07 | 75.80 | 67.39 | 67.46 | 69.66 | 85.78 |
| DNABERT-2 (BPE) | 117M | 262B | 71.99 | 76.06 | 66.52 | 58.54 | 77.43 | 86.77 | 94.27 | 71.59 | 69.37 | 68.04 | 74.17 | 84.99 |
| VQDNA (HRQ) | 103M | 262B | 72.48 | 76.43 | 66.85 | 58.92 | 78.10 | 90.75 | 94.48 | 74.52 | 71.02 | 70.58 | 78.50 | 89.53 |
| BIOARC (mask-ft) | 4.89M | 3.1B | **84.80** | 86.00 | 85.80 | 77.10 | 89.20 | 92.62 | **96.55** | 83.20 | **83.60** | **85.43** | 89.40 | 90.84 |
| BIOARC (con-ft) | 3.28M | 3.1B | **84.80** | **86.10** | **86.50** | **77.50** | **89.30** | **93.12** | 96.25 | **83.69** | 83.53 | 84.66 | **90.05** | **91.06** |
| BIOARC (ntp-ft) | 6.58M | 3.1B | 84.20 | 85.90 | 82.80 | 75.50 | 89.10 | 91.82 | 96.25 | 76.02 | 82.26 | 84.38 | 74.55 | 83.76 |

*Table 2.* Performance on different protein tasks. The **top block** reports reference baselines from released checkpoints (much larger pretraining budgets). The **bottom block** is a controlled comparison in which BIOARC and ESM-2 are pretrained under identical conditions (full UniRef50, 50K steps) to isolate architecture; **bold** marks the better model within this controlled comparison. ESM-2 8M (official) is the released 8M checkpoint (full UniRef50, 500K steps), shown for reference. The corresponding pretraining loss is reported in Appendix A.6.5.

| Method | #Param | #Data | Solubility | HumanPPI | PPIAffinity | Fold | Subcellular | Binary |
|---|---|---|---|---|---|---|---|---|
| ProtBert | 419.9M | 393B | 68.15 | 77.32 | 2.195 | 16.94 | 76.53 | 91.32 |
| ESM-1b | 652.4M | 86B | 70.23 | 78.17 | 2.281 | 28.17 | 78.13 | 92.40 |
| ESM2-1b | 652.4M | 86B | 74.13 | 80.49 | 2.134 | 32.31 | 80.45 | 92.79 |
| ESM-2 8M (official) | 8M | 86B | 73.48 | 80.16 | 3.098 | 22.14 | 71.47 | 91.25 |
| *Controlled comparison (identical pretraining: full UniRef50, 50K steps)* | | | | | | | | |
| ESM-2 8M (reimpl.) | 8M | 50K steps | 71.84 | 74.68 | 3.567 | 18.25 | 70.36 | 90.68 |
| BIOARC 8M | 8M | 50K steps | **73.29** | **76.79** | **2.756** | **20.75** | **72.77** | **91.82** |

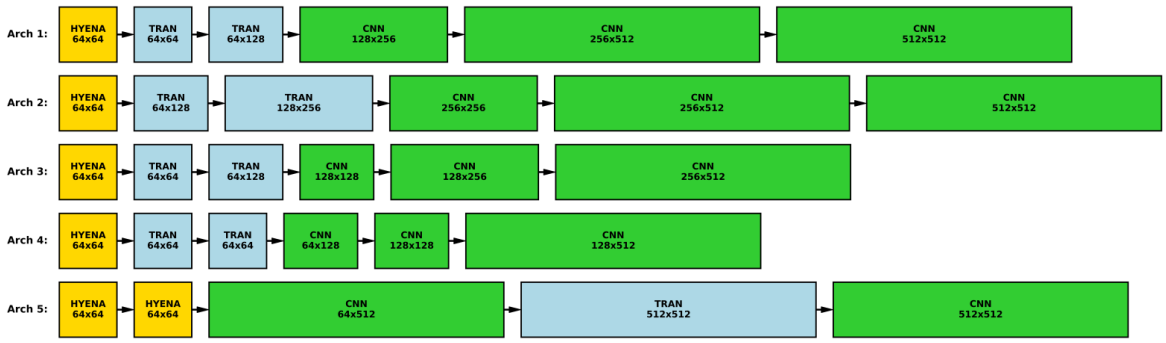



*Figure 2.* This figure illustrates the top five performing DNA model architectures (Arch 1-5), identified by averaging their performance across the different tasks. Results on protein is shown in Appendix A.6.1.

found in Appendix A.7.1.

**Layer-wise Contribution Analysis** To demonstrate the synergistic efficacy of our hybrid modules, we conducted probing experiments by attaching classifiers to the output of each layer. We find that the model achieves consistent monotonic improvements in accuracy layer-by-layer, verifying that the specific sequence of diverse modules collaboratively refines biological representations rather than relying on a single dominant layer. Details can be found in Appendix A.7.2.

**Architecture Depth Analysis** To verify the scalability of our discovered patterns, we investigated the impact of network depth on transfer performance. We find that while deeper models generally offer higher potential capacity, performance is not solely determined by depth; however, once an optimal architectural pattern is identified, scaling up the depth consistently enhances robustness. Details can be found in Appendix A.7.3.

**Performance-Parameter Analysis** To confirm that the superiority of our models stems from topological efficiency rather than mere capacity, we analyzed the relationship between parameter count and accuracy across the search space. We find that there is no positive correlation between model size and performance; instead, specific hybrid topologies (inductive biases) are the decisive factors for success, achieving high accuracy with significantly fewer parameters. Details can be found in Appendix A.7.4.

**Architecture Ranking Correlation Analysis** To validate the reliability of our one-shot search strategy, we evalu-

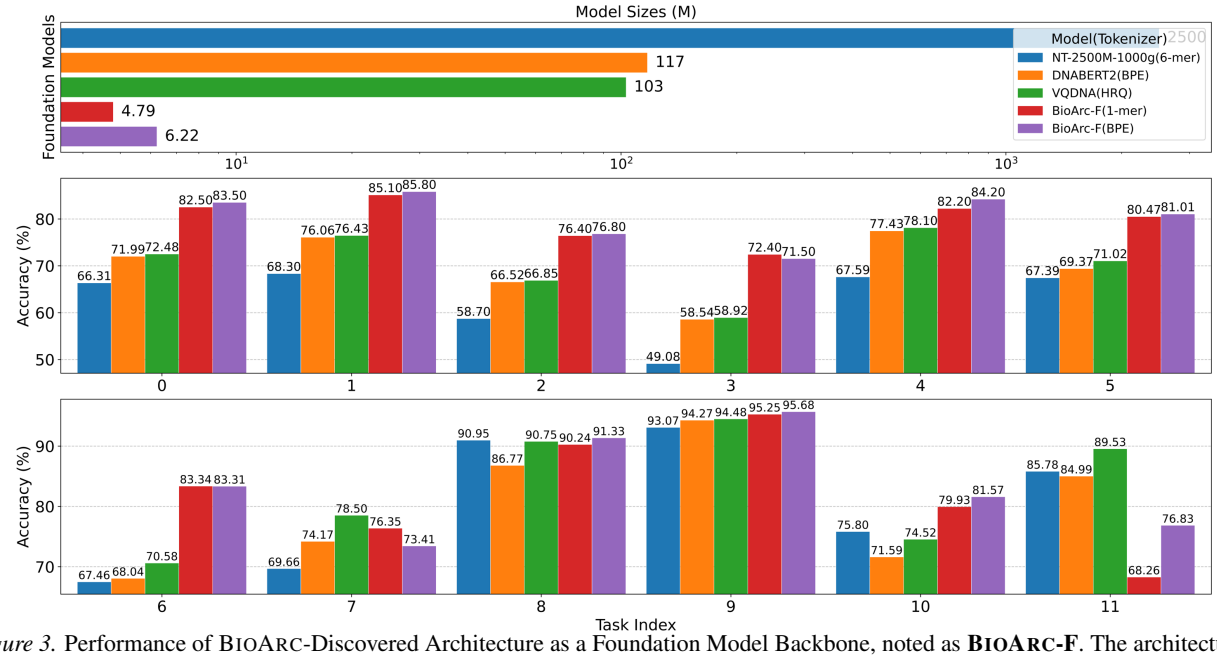



*Figure 3.* Performance of BIOARC-Discovered Architecture as a Foundation Model Backbone, noted as **BIOARC-F**. The architecture is selected by the top average performance across all tasks and pretrained for 1/10 training steps of baselines.

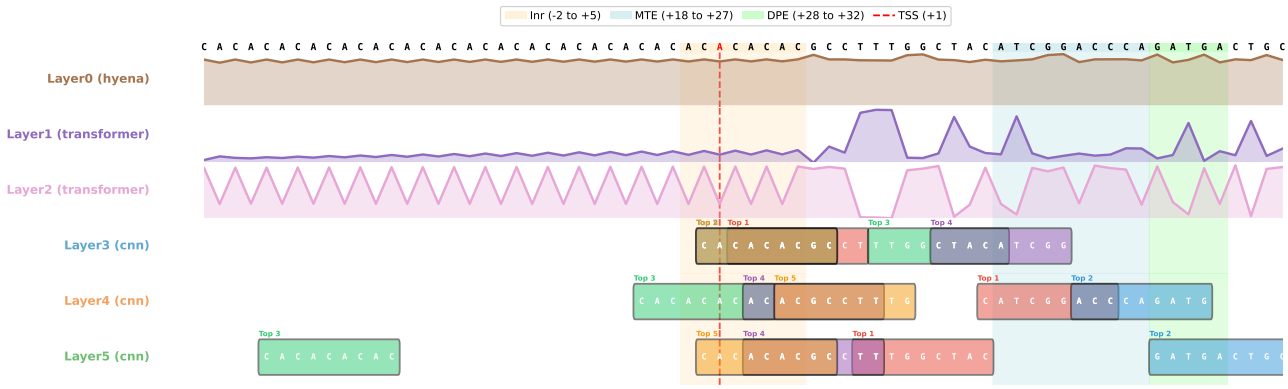



*Figure 4.* **Hierarchical Interpretation of Promoter Grammar. Layer 0 (Hyena):** Extracts global genomic context via continuous activations. **Layers 1-2 (Transformer):** Maps spatial syntax; Layer 1 anchors downstream elements while Layer 2 identifies the TSS functional boundary via pattern transition. **Layers 3-5 (CNN):** Acts as a morphological decoder, transitioning from localized motif detection (L3-4) to multi-aspect integration of the Inr-MTE-DPE complex (L5). Details are provided in Appendix A.8.

ated the consistency of architecture rankings. First, we observed a low correlation (Spearman's $\rho = 0.3222$) between the rankings derived from the ***pretrained and finetuned supernet*** versus those ***trained from scratch***. Figure 14 indicates that finetuning architectures individually is essential. Second, rankings derived from the ***pretrained supernet with individual finetuning*** exhibit a strong positive correlation (Spearman's $\rho = 0.7287$) with the ***train-from-scratch*** baseline. As illustrated in Figure 15, validating that the architectural rankings remain consistent regardless of the pretraining initialization. Further details are provided in Appendix A.7.5.

**Architecture Prediction.** To extend our method to unseen tasks without repetitive search, we leveraged the observation that similar tasks consistently favor similar optimal topologies. By benchmarking predictors based on traditional neural networks, standard LLMs, and autonomous agents, we find that the agent-based system achieves the highest performance in inferring optimal designs, effectively bridging the gap between offline search and real-world deployment. Details can be found in Appendix A.7.7.

## 5. Conclusion

We introduce BIOARC, a neural architecture search framework. By exploring a vast space, we identify high-

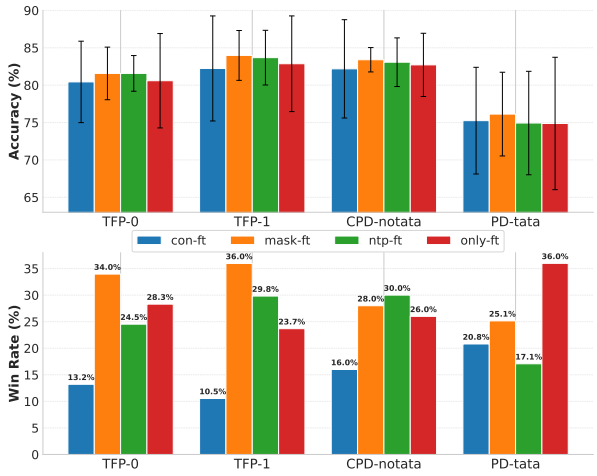
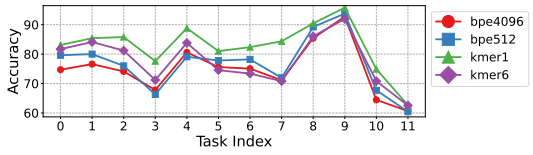
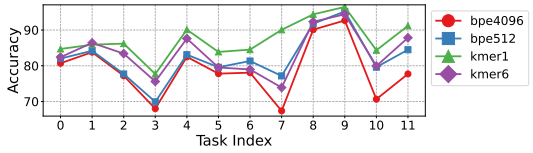



*Figure 5.* Performance of different training strategies on DNA. In the top panel, each cell shows the mean performance ± standard deviation across all architectures. In the bottom panel, each bar shows the percentage of the total 360 architectures that chose that training strategy to yield the best performance. More results on Protein could be found in Appendix A.6.6.




*(a)* CNN




*(b)* Hybrid architectures

*Figure 6.* Effect of different tokenization on various architectures training from scratch on the DNA tasks. Detailed analysis across modalities and training strategies is extended in Appendix A.6.7.

performing foundational and task-specific architectures that often outperform larger established models. We also analyze how tokenization and training interact with architecture, clarifying their critical impact. Furthermore, fine-grained interpretability demonstrates that the discovered hybrid architectures capture biological grammar, validating our design's biological relevance. Ultimately, BIOARC provides a foundational resource and principled methodology for creating the next generation of biology-tailored models.

## Acknowledgements

This research is sponsored by NSF #2442253, Commonwealth Cyber Initiative, and generous gifts from Nvidia, Cisco, and the Amazon-Virginia Tech Initiative. This research used the Delta system at the National Center for Supercomputing Applications [award OAC 2005572] through allocation [NAIRR240202] from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296. This research is also partially supported by allocation [NAIRR240439].

## Impact Statement

This paper presents work whose goal is to advance the field of biological foundation model. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

## References

Akiyama, M. and Sakakibara, Y. Informative rna base embedding for rna structural alignment and clustering by deep representation learning. *NAR Genomics and Bioinformatics*, 4(1):lqac012, 02 2022. ISSN 2631-9268. doi: 10.1093/nargab/lqac012. URL https://doi.org/10.1093/nargab/lqac012.

Avsec, Ž., Agarwal, V., Visentin, D., Lajić, S., Curk, T., and Gjorgjieva, E. Effective gene expression prediction from sequence by integrating long-range interactions. *Nature Methods*, 18(10):1196–1203, 2021. doi: 10.1038/s41592-021-01252-x. URL https://doi.org/10.1038/s41592-021-01252-x.

Bender, G., Kindermans, P.-J., Zoph, B., Vasudevan, V., and Le, Q. V. Understanding and simplifying one-shot architecture search. In *International Conference on Machine Learning*, 2018. URL https://api.semanticscholar.org/CorpusID:51763580.

Brixi, G., Durrant, M. G., Ku, J., Poli, M., Brockman, G., Chang, D., Gonzalez, G. A., King, S. H., Li, D. B., Merchant, A. T., Naghipourfar, M., Nguyen, E., Ricci-Tam, C., Romero, D. W., Sun, G., Taghibakshi, A., Vorontsov, A., Yang, B., Deng, M., Gorton, L., Nguyen, N., Wang, N. K., Adams, E., Baccus, S. A., Dillmann, S., Ermon, S., Guo, D., Ilango, R., Janik, K., Lu, A. X., Mehta, R., Mofrad, M. R., Ng, M. Y., Pannu, J., Ré, C., Schmok, J. C., John, J. S., Sullivan, J., Zhu, K., Zynda, G., Balsam, D., Collison, P., Costa, A. B., Hernandez-Boussard, T., Ho, E., Liu, M.-Y., McGrath, T., Powell, K., Burke, D. P., Goodarzi, H., Hsu, P. D., and Hie, B. L. Genome modeling and design across all domains of life with evo 2. *bioRxiv*, 2025. doi: 10.1101/2025.02.18.638918. URL https://www.biorxiv.org/content/early/2025/02/21/2025.02.18.638918.

Burke, T. W. and Kadonaga, J. T. Drosophila tfiid binds to a conserved downstream basal promoter element that is present in many tata-box-deficient promoters. *Genes Dev.*,

10(6):711–724, Mar 1996. doi: 10.1101/gad.10.6.711. PMID: 8598298.

Burke, T. W. and Kadonaga, J. T. The downstream core promoter element, DPE, is conserved from Drosophila to humans and is recognized by TAFII60 of Drosophila. *Genes Dev*, 11(22):3020–3031, Nov 1997. doi: 10.1101/gad.11.22.3020.

Consortium, T. U. The universal protein resource (uniprot). *Nucleic Acids Research*, 36(suppl_1):D190–D195, 11 2007. ISSN 0305-1048. doi: 10.1093/nar/gkm895. URL https://doi.org/10.1093/nar/gkm895.

Dalla-Torre, H., Gonzalez, L., Mendoza-Revilla, J., Carranza, N. L., Grzywaczewski, A. H., Oteri, F., Dallago, C., Trop, E., Sirelkhatim, H., Richard, G., Skwark, M., Beguir, K., Lopez, M., and Pierrot, T. The nucleotide transformer: Building and evaluating robust foundation models for human genomics. *bioRxiv*, 2023. doi: 10.1101/2023.01.11.523679.

Deaton, A. M. and Bird, A. Cpg islands and the regulation of transcription. *Genes & development*, 25(10):1010–1022, 2011.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 248–255. IEEE, 2009.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL https://arxiv.org/abs/1810.04805.

Dill, K. A. and MacCallum, J. L. The protein-folding problem, 50 years on. *Science*, 338(6110):1042–1046, 2012. doi: 10.1126/science.1219021.

Dip, S. A., Shuvo, U. A., Chau, T., Song, H., Choi, P., Wang, X., and Zhang, L. Patholm: Identifying pathogenicity from the dna sequence through the genome foundation model, 2024. URL https://arxiv.org/abs/2406.13133.

Dosovitskiy, A. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

Duan, Y., Chen, X., Xu, H., Chen, Z., Liang, X., Zhang, T., and Li, Z. Transnas-bench-101: Improving transferability and generalizability of cross-task neural architecture search, 2021. URL https://arxiv.org/abs/2105.11871.

Eisenstein, M. Foundation models build on ChatGPT tech to learn the fundamental language of biology. *Nature Biotechnology*, 42(9):1323–1325, Sep 2024. doi: 10.1038/s41587-024-02400-2. URL https://doi.org/10.1038/s41587-024-02400-2.

Elnaggar, A., Heinzinger, M., Dallago, C., Rehawi, G., Wang, Y., Jones, L., Gibbs, T., Feher, T., Angerer, C., Steinegger, M., Bhowmik, D., and Rost, B. Prottrans: Towards cracking the language of life's code through self-supervised learning. *bioRxiv*, 2021. doi: 10.1101/2020.07.12.199554. URL https://www.biorxiv.org/content/early/2021/05/04/2020.07.12.199554.

Fishman, V., Kuratov, Y., Petrov, M., Shmelev, A., Shepelin, D., Chekanov, N., Kardymon, O., and Burtsev, M. Gena-lm: A family of open-source foundational models for long dna sequences. *bioRxiv*, 2023. doi: 10.1101/2023.06.12.544594. URL https://www.biorxiv.org/content/early/2023/06/13/2023.06.12.544594.

Greener, J. G., Kandathil, S. M., Moffat, L., and Jones, D. T. A guide to machine learning for biologists. *Nature Reviews Molecular Cell Biology*, 23(1):40–55, Jan 2022. doi: 10.1038/s41580-021-00407-0. URL https://doi.org/10.1038/s41580-021-00407-0.

Grešová, K., Martinek, V., Čechák, D., Šimeček, P., and Alexiou, P. Genomic benchmarks: a collection of datasets for genomic sequence classification. *BMC Genomic Data*, 24(1):25, 2023. doi: 10.1186/s12863-023-01123-8.

Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=AL1Oo7_aKC.

Gündüz, H. A., Mreches, R., Moosbauer, J., Robertson, G., To, X. Y., Franzosa, E. A., Huttenhower, C., Rezaei, M., McHardy, A. C., Bischl, B., Münch, P. C., and Binder, M. Optimized model architectures for deep learning on genomic data. *Communications Biology*, 7(1):516, Apr 2024. doi: 10.1038/s42003-024-06161-1. Erratum in: Commun Biol. 2024 May 23;7(1):625. doi: 10.1038/s42003-024-06318-y.

Guo, Z., Zhang, X., Mu, H., Heng, W., Liu, Z., Wei, Y., and Sun, J. Single path one-shot neural architecture search with uniform sampling, 2020. URL https://arxiv.org/abs/1904.00420.

Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models, 2020. URL https://arxiv.org/abs/2006.11239.

Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

Javahery, R., Khachi, A., Lo, K., Zenzie-Gregory, B., and Smale, S. T. Dna sequence requirements for transcriptional initiator activity in mammalian cells. *Mol Cell Biol*, 14(1):116–127, 1994. doi: 10.1128/mcb.14.1.116-127.1994.

Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021.

Krizhevsky, A., Sutskever, I., and Hinton, G. E. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, volume 25, 2012.

Kutach, A. K. and Kadonaga, J. T. The downstream promoter element dpe appears to be as widely used as the tata box in drosophila core promoters. *Molecular and Cellular Biology*, 20(13):4754–4764, 2000.

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., and Soricut, R. Albert: A lite bert for self-supervised learning of language representations, 2020. URL https://arxiv.org/abs/1909.11942.

Li, C., Tang, T., Wang, G., Peng, J., Wang, B., Liang, X., and Chang, X. Bossnas: Exploring hybrid cnn-transformers with block-wisely self-supervised neural architecture search, 2021. URL https://arxiv.org/abs/2103.12424.

Li, S., Wang, Z., Liu, Z., Wu, D., Tan, C., Zheng, J., Huang, Y., and Li, S. Z. Vqdna: Unleashing the power of vector quantization for multi-species genomic sequence modeling, 2024. URL https://arxiv.org/abs/2405.10812.

Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., Smetanin, N., Verkuil, R., Kabeli, O., Shmueli, Y., dos Santos Costa, A., Fazel-Zarandi, M., Sercu, T., Candido, S., and Rives, A. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574. URL https://www.science.org/doi/abs/10.1126/science.ade2574.

Liu, H., Simonyan, K., and Yang, Y. Darts: Differentiable architecture search, 2019. URL https://arxiv.org/abs/1806.09055.

Liu, H., Li, C., Wu, Q., and Lee, Y. J. Visual instruction tuning. *Advances in neural information processing systems*, 36, 2024.

Ma, L., Cui, J., and Yang, B. Deep neural architecture search with deep graph bayesian optimization, 2019. URL https://arxiv.org/abs/1905.06159.

Marcos, E., Basanta, B., Chidyausiku, T. M., Tang, Y., Oberdorfer, G., Liu, G., Swapna, G., Guan, R., Silva, D.-A., Dou, J., Pereira, J. H., Xiao, R., Sankaran, B., Zwart, P. H., Montelione, G. T., and Baker, D. Principles for designing proteins with cavities formed by curved β-sheets. *Science*, 355(6321):201–206, 2017. doi: 10.1126/science.aah7383.

Microsoft. Neural Network Intelligence, January 2021. URL https://github.com/microsoft/nni.

Nguyen, E., Poli, M., Durrant, M. G., Kang, B., Katrekar, D., Li, D. B., Bartie, L. J., Thomas, A. W., King, S. H., Brixi, G., Sullivan, J., Ng, M. Y., Lewis, A., Lou, A., Ermon, S., Baccus, S. A., Hernandez-Boussard, T., Ré, C., Hsu, P. D., and Hie, B. L. Sequence modeling and design from molecular to genome scale with evo. *Science*, 386(6723):eado9336, 2024. doi: 10.1126/science.ado9336. URL https://www.science.org/doi/abs/10.1126/science.ado9336.

OpenAI, R. Gpt-4 technical report. *arXiv*, pp. 2303–08774, 2023.

Poli, M., Massaroli, S., Nguyen, E., Fu, D. Y., Dao, T., Baccus, S. A., Yoshimoto, Y., Clément, C., López-Paz, D., and Ré, C. Hyena hierarchy: Towards larger convolutional language models. In *International Conference on Machine Learning (ICML)*, pp. 27926–27950. PMLR, 2023.

Real, E., Moore, S., Selle, A., Saxena, S., Suematsu, Y. L., Tan, J., Le, Q., and Kurakin, A. Large-scale evolution of image classifiers, 2017. URL https://arxiv.org/abs/1703.01041.

Ren, J., Song, K., Deng, C., Ahlgren, N. A., Fuhrman, J. A., Li, Y., Xie, X., Poplin, R., and Sun, F. Identifying viruses from metagenomic data using deep learning. *Quantitative Biology*, 8(1):64–77, Mar 2020. doi: 10.1007/s40484-019-0187-4.

Rives, A., Meier, J., Sercu, T., Goyal, S., Lin, Z., Liu, J., Guo, D., Ott, M., Zitnick, C. L., Ma, J., and Fergus, R. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15):e2016239118, 2021. doi: 10.1073/pnas.2016239118. URL https://www.pnas.org/doi/abs/10.1073/pnas.2016239118.

Sapoval, N., Aghazadeh, A., Nute, M. G., Moore, F. H., Gob Biswas, S. D., Wang, M. F. Z., and AlQuraishi,

M. Current progress and open challenges for applying deep learning across the biosciences. *Nature Communications*, 13(1):1728, 2022. doi: 10.1038/s41467-022-29268-7. URL https://doi.org/10.1038/s41467-022-29268-7.

Schiff, Y., Kao, C.-H., Gokaslan, A., Dao, T., Gu, A., and Kuleshov, V. Caduceus: Bi-directional equivariant long-range dna sequence modeling. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2403.03234.

Sennrich, R., Haddow, B., and Birch, A. Neural machine translation of rare words with subword units, 2016. URL https://arxiv.org/abs/1508.07909.

Sivangi, K. B., Dasari, C. M., Amilpur, S., and Bhukya, R. Noas-ds: Neural optimal architecture search for detection of diverse dna signals. *Neural Networks*, 147:63–71, 2022. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2021.12.009. URL https://www.sciencedirect.com/science/article/pii/S0893608021004822.

Steinegger, M., Mirdita, M., and Söding, J. Protein-level assembly increases protein sequence recovery from metagenomic samples manyfold. *Nature Methods*, 16(7):603–606, 2019. doi: 10.1038/s41592-019-0437-4.

Suzek, B. E., Huang, H., McGarvey, P., Mazumder, R., and Wu, C. H. Uniref: comprehensive and non-redundant uniprot reference clusters. *Bioinformatics*, 23(10):1282–1288, may 2007. ISSN 1367-4803. doi: 10.1093/bioinformatics/btm098.

Tan, M. and Le, Q. V. Efficientnet: Rethinking model scaling for convolutional neural networks, 2020. URL https://arxiv.org/abs/1905.11946.

Theodoris, C. V., Xiao, L., Chopra, A., Chiorazzi, M. H., Retkwa, R. S. F., Varnavides, G., Hsieh, E. M., Vaziri, S. A., Lee, V. S., Ward, C. W., He, J. D. R. D., Aalfs, J. R., Fonoudi, H., Pico, A., Gambhir, S. S., Snyder, M. P., Pollard, K. A., and Srivastava, D. Transfer learning enables predictions in network biology. *Nature*, 618(7995):616–624, Jun 2023. doi: 10.1038/s41586-023-06139-9. URL https://doi.org/10.1038/s41586-023-06139-9.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models, 2023. URL https://arxiv.org/abs/2302.13971.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Vishniakov, K., Viswanathan, K., Medvedev, A., Kanithi, P., Pimentel, M. A., Rajan, R., and Khan, S. Genomic foundationless models: Pretraining does not promise performance. *bioRxiv*, 2025. doi: 10.1101/2024.12.18.628606. URL https://www.biorxiv.org/content/early/2025/06/25/2024.12.18.628606.

Wang, N., Bian, J., Li, Y., Wang, D., Duan, J., Wan, Y., Wang, J., Ma, J., and Li, Y.-f. Multi-purpose RNA language modelling with motif-aware pretraining and type-guided fine-tuning. *Nature Machine Intelligence*, 6(5):548–557, 2024. doi: 10.1038/s42256-024-00836-4. URL https://doi.org/10.1038/s42256-024-00836-4.

Wang, S., Sun, S., Li, Z., Zhang, R., and Xu, J. Accurate de novo prediction of protein contact map by ultra-deep learning model. *PLOS Computational Biology*, 13(1):e1005324, January 2017. ISSN 1553-7358. doi: 10.1371/journal.pcbi.1005324. URL http://dx.doi.org/10.1371/journal.pcbi.1005324.

White, C., Zela, A., Ru, B., Liu, Y., and Hutter, F. How powerful are performance predictors in neural architecture search?, 2021. URL https://arxiv.org/abs/2104.01177.

Wu, W., Li, Q., Li, M., Fu, K., Feng, F., Ye, J., Xiong, H., and Wang, Z. Generator: A long-context generative genomic foundation model, 2025. URL https://arxiv.org/abs/2502.07272.

Xiao, Y., Zhao, W., Zhang, J., Jin, Y., Zhang, H., Ren, Z., Sun, R., Wang, H., Wan, G., Lu, P., Luo, X., Zhang, Y., Zou, J., Sun, Y., and Wang, W. Protein large language models: A comprehensive survey, 2025. URL https://arxiv.org/abs/2502.17504.

Xu, M., Zhang, Z., Lu, J., Zhu, Z., Zhang, Y., Ma, C., Liu, R., and Tang, J. Peer: A comprehensive and multi-task benchmark for protein sequence understanding, 2022. URL https://arxiv.org/abs/2206.02096.

Ying, C., Klein, A., Real, E., Christiansen, E., Murphy, K., and Hutter, F. Nas-bench-101: Towards reproducible neural architecture search, 2019. URL https://arxiv.org/abs/1902.09635.

Yu, Z., Liu, F., and Li, Y. sctca: a hybrid transformer-cnn architecture for imputation and denoising of scdna-seq data. *Briefings in Bioinformatics*, 25(6):bbae577, 11 2024. ISSN 1477-4054. doi: 10.1093/bib/bbae577. URL https://doi.org/10.1093/bib/bbae577.

Zeng, Y., Xie, J., Shangguan, N., et al. Cellfm: a large-scale foundation model pre-trained on transcriptomics of 100 million human cells. *Nat Commun*, 16:4679, May 2025. doi: 10.1038/s41467-025-59926-5.

Zhang, Z., Cofer, E. M., and Troyanskaya, O. G. Ambient: Accelerated convolutional neural network architecture search for regulatory genomics. *bioRxiv*, 2021a. doi: 10.1101/2021.02.25.432960. URL https://www.biorxiv.org/content/early/2021/02/27/2021.02.25.432960.

Zhang, Z., Park, C. Y., Theesfeld, C. L., et al. An automated framework for efficiently designing deep convolutional neural networks in genomics. *Nature Machine Intelligence*, 3(5):392–400, May 2021b. doi: 10.1038/s42256-021-00316-z.

Zhou, Z., Ji, Y., Li, W., Dutta, P., Davuluri, R., and Liu, H. Dnabert-2: Efficient foundation model and benchmark for multi-species genome, 2024. URL https://arxiv.org/abs/2306.15006.

Zoph, B. and Le, Q. V. Neural architecture search with reinforcement learning, 2017. URL https://arxiv.org/abs/1611.01578.

Łukasz Dudziak, Chau, T., Abdelfattah, M. S., Lee, R., Kim, H., and Lane, N. D. Brp-nas: Prediction-based nas using gcns, 2021. URL https://arxiv.org/abs/2007.08668.

# A. Appendix

## A.1. Benchmark Details

### A.1.1. DNA

**Transcription Factor Prediction (TFP, Task 0-4):** This task involves predicting transcription factor binding sites (TFBS) in the human genome using data derived from 690 ENCODE ChIP-seq experiments.

**Core Promoter Detection (CPD, Task 5-7):** This task involves identifying the precise location of core promoter regions, which are essential for initiating gene transcription, focusing on predicting the core promoter region only, the central region closest to the TSS and start codon. A much shorter context window (center -34 +35 bp around TSS) is provided, making this a more challenging task than proximal promoter prediction.

**Promoter Detection (PD, Task 8-10):** This task requires the model to identify human proximal promoter regions, which contain primary regulatory elements crucial for transcription initiation. We construct three dataset variations: TATA, non-TATA, and a combined All set. Positive samples are extracted from the Eukaryotic Promoter Database (EPDnew) (Dreos et al., 2013), covering the window from -249 to +50 bp relative to the Transcription Start Site (TSS). Negative control strategies differ by subset: the TATA dataset uses random non-promoter genomic sequences containing TATA motifs, while the non-TATA dataset utilizes randomly substituted sequences to represent the null class.

**Splice Site Prediction (SSP, Task 11):** This task focuses on splice site prediction in the human genome, a process vital for understanding protein diversity and genetic diseases.

### A.1.2. PROTEIN

**Solubility Prediction (Task 12):** a binary classification task that determines if a protein is soluble or insoluble. To ensure models can generalize to new and dissimilar proteins, the dataset is split so that the training proteins have less than 30% sequence identity with the test proteins. This is a critical task because good solubility is an essential property for functional proteins, especially in pharmaceutical research and industry. The ultimate goal is to drive the development of more effective computational tools that can predict a protein's solubility based solely on its amino acid sequence.

**Human PPI Prediction (Task 13):** A binary classification task to determine whether a pair of human proteins will physically interact. The data is split into an 8:1:1 ratio for training, validation, and testing, and the task is designed to test how well models can generalize to dissimilar protein sequences. The work is significant because understanding the human protein interactome is vital for deciphering disease mechanisms. This task serves as a benchmark to drive the development of more effective machine learning models for PPI prediction.

**Fold Classification (Task 14):** A classification task that predicts the overall 3D structural shape (fold) of a protein from 1,195 possible categories. The model is specifically tested on its ability to perform remote homology detection, meaning it must recognize proteins with similar structures even if their sequences are very different. To achieve this, entire superfamilies of proteins are withheld from the training data and used only for testing. The goal is to automate fold classification using protein sequences, which is important for drug design and functional analysis because most known protein structures have not yet been manually classified.

**PPI Affinity Prediction (Task 15):** A regression task that estimates the binding strength between two proteins. Using the SKEMPI dataset, the data is split based on the number of mutations to test the model's generalization capabilities in a scenario mimicking multi-round protein engineering: Training set is wild-type proteins and mutants with less then 2 mutations. Validation set is mutants with 3 or 4 mutations. Test set is mutants with more then 4 mutations. The primary impact of this task is its direct application to protein binder design, where accurately predicting the binding strength of candidate molecules is crucial for developing new therapeutics and biotechnologies.

**Subcellular Location Prediction (Task 16):** A multiclass classification task involves predicting which of 10 possible locations a protein resides in within a cell. It is a multi-class classification problem where models are trained and tested using the DeepLoc dataset, which is specifically partitioned to evaluate performance on homologous proteins. The primary impact of this research is in drug discovery. Knowing a protein's location helps identify it as a potential drug target, and an accurate, high-throughput prediction tool can significantly accelerate this process.

**Binary Location Prediction (Task 17):** A simplified version of subcellular localization, framed as a binary classification problem designed to classify proteins as either membrane-bound or soluble, using a binary label like 0 or 1. The model is

trained and tested on data from the DeepLoc dataset, with a key evaluation being its ability to generalize and make accurate predictions for homologous (structurally similar) proteins. The task is significant because it helps efficiently distinguish between soluble proteins, which are free-floating, and membrane-bound proteins, which are attached to cell membranes and can have important catalytic functions.

### A.2. Modules Designs

In this section, we provide the detailed architectural specifications for the various building blocks employed in our framework. Table 3 presents a summary of these designs, outlining the core layers, normalization and activation mechanisms, as well as the specific hyperparameter settings (e.g., kernel sizes, initialization schemes) used in our experiments.

*Table 3.* Detailed Specifications of Architectural Modules

| Module | Core Layer | Norm / Activation | Key Hyperparameters |
|---|---|---|---|
| **CNN** | Conv1d ($D_{in} \to D_{out}$) | ReLU $\to$ BatchNorm1d | Kernel: $\{5, 9\}$, Padding: $\{2, 4\}$ |
| **Hyena** | HyenaEncoderLayer | *None* (Identity) | Proj: Linear ($D_{in} \to D_{out}$) optional; Model Dim: $D_{out}$ |
| **Transformer** | TransformerEncoderLayer | LayerNorm (internal) | Heads: $D_{out}/64$; Input matched to $D_{out}$ |
| **Mamba** | Mamba Layer | LayerNorm (pre-block) | Proj: Linear ($D_{in} \to D_{out}$) optional |
| **LSTM** | LSTM (1 layer) | Sigmoid/Tanh (internal gates) | Hidden: $D_{out}$, Dropout: 0.4, Init: Orthogonal (rec), Xavier (in), Bias 0 |

### A.3. Search Space Pruning

The combinatorial nature of search space design, while comprehensive, leads to an exponential growth in the number of candidate architectures as the number of choices increases. To maintain a computationally tractable search space without sacrificing diversity, we employ three strategies to select a representative subset of configurations for both module types ($\mathcal{C}^{(d)}_{\text{type}}$) and hidden dimensions ($\mathcal{C}^{(d)}_{\text{dim}}$) at each depth $d \in \mathcal{D}$ (in Equation 1).

**Rule-based pruning.** We enforces a monotonic non-decreasing width constraint (i.e., $h_i \geq h_{i-1}$ for $i \geq 1$) to encourage progressive feature extraction.

**Distance-based pruning.** We first generate all possible valid dimension paths that satisfy the non-decreasing constraint. To ensure diversity from the outset, we employ a greedy selection algorithm. A candidate path is only added to our initial representative set if it is sufficiently distant from all previously selected paths. The distance between two dimension paths, $\mathbf{h}_a$ and $\mathbf{h}_b$, is measured as the Euclidean distance in a **log-transformed space**. Specifically, given full paths $\mathbf{h}'_a = (h_0, h_{1,a}, \ldots, h_{d,a})$ and $\mathbf{h}'_b = (h_0, h_{1,b}, \ldots, h_{d,b})$, where $h_0$ is the fixed embedding dimension, their distance is:

$$D(\mathbf{h}'_a, \mathbf{h}'_b) = \sqrt{\sum_{i=0}^{d} \left(\log_2(h_{i,a}) - \log_2(h_{i,b})\right)^2}$$

A candidate path $\mathbf{h}_{\text{cand}}$ is kept only if $D(\mathbf{h}'_{\text{cand}}, \mathbf{h}'_{\text{rep}}) \geq \tau$ for all paths $\mathbf{h}_{\text{rep}}$ already in the representative set, where $\tau$ is a predefined distance threshold. Using a log scale ensures that the selection is sensitive to relative changes in dimension (e.g., distinguishing between 64 and 128) rather than absolute differences.

**Cluster-based pruning.** We deploy clustering method in both module types and hidden dimension orders. For a given depth $d$, the total number of possible module type configurations is $|\mathcal{M}|^d$. To reduce this number to a manageable target $k_d$, we perform clustering to identify a set of diverse and representative paths. The procedure is as follows:

1. **Vector Representation**: Each module type path $\mathbf{m} = (m_1, m_2, \ldots, m_d)$ is first transformed into a numerical vector. We apply **one-hot encoding** to each module $m_i \in \mathcal{M}$, converting the categorical sequence into a high-dimensional numerical representation. This results in a flattened vector for each path.
2. **K-Means Clustering**: We then apply the K-Means clustering algorithm to the set of all one-hot encoded vectors. The number of clusters is set to the desired number of representative configurations $k_d$. K-Means partitions the paths into $k_d$ clusters by minimizing the within-cluster sum of squares.
3. **Centroid-based Selection**: The centroid of each cluster represents the mean of the paths within it. Since a centroid may not correspond to a valid, discrete module type path, we select the actual path from the dataset that is closest to each cluster centroid in terms of Euclidean distance. This ensures that our final set $\mathcal{C}_{\text{type}}^{(d)}$ consists of $k_d$ valid and diverse module type configurations.

The search space for hidden dimension paths $\mathbf{h} = (h_1, h_2, \ldots, h_d)$ is also vast. We apply the same pipeline on it, except changing the first encoding step.

By applying these structured reduction techniques to both the module type and hidden dimension configurations, we construct a final search space $\mathcal{A}$ that is both diverse and computationally feasible, allowing for an efficient yet comprehensive exploration of the architectural landscape.

### A.4. Training Strategies

**Masked Modeling (MM)** For a given input sequence $S = (x_1, x_2, \ldots, x_L)$, we randomly mask a fraction of its tokens to create a corrupted sequence $\tilde{S}$. The supernet is then tasked with predicting the original tokens for the masked positions based on the contextual information provided by the unmasked tokens. This process compels the model to learn intricate local dependencies and contextual relationships within biological sequences.

$$\mathcal{L}_{\text{MM}} = -\sum_{t \in \mathcal{M}} \log p(x_t \mid \tilde{S})$$

where $\mathcal{M}$ is the set of indices of the masked tokens.

**Contrastive Learning (CL)** This approach aims to learn an embedding space where representations of similar sequences are pulled closer together, while those of dissimilar sequences are pushed apart. For a batch of sequences $\{S_i\}_{i=1}^{B}$, we generate a "positive" pair for each sequence $S_i$ by applying data augmentations, resulting in encoded representations $z_i$ and $z_j$. All other sequences in the batch are treated as "negative" examples. The model is trained to maximize the cosine similarity of positive pairs while minimizing it for negative pairs:

$$\mathcal{L}_{\text{CL}} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}$$

where $\text{sim}(\cdot)$ denotes cosine similarity, and $\tau$ is a temperature hyperparameter.

**Next Token Prediction (NTP)** For a given input sequence $S = (x_1, x_2, \ldots, x_L)$, this objective leverages the auto-regressive nature of biological sequences. The supernet is tasked with predicting the next token $x_{t+1}$ based on the preceding context in $S$. This process compels the model to learn the sequential grammar and causal dependencies:

$$\mathcal{L}_{\text{NTP}} = -\sum_{t=1}^{L-1} \log p(x_{t+1} \mid x_1, \ldots, x_t)$$

where $L$ is the length of the sequence $S$.

### A.5. Experiments Settings

#### A.5.1. DEFAULT TRAINING SETTINGS

**For supernet pretraining,** we used NNI framework implementation (Microsoft, 2021). We use one A100 80G for the pretraining for 10 epoch.

*Table 4.* Hyperparameters for Individual Task Training

| Modality | Learning Rate | Batch Size | Epoch | Num Warmup Steps | Weight Decay |
|---|---|---|---|---|---|
| DNA | $3 \times 10^{-5}$ | 32 | 3(task 0-5) 10 (task 6-11) | 50 | 0.01 |
| Protein | $5 \times 10^{-5}$ | 32 | 5 | 50 | 0.01 |

**For overall best architecture pretraining in Figure 3**, we use AdamW optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 1e-6$ and weight decay of $1e-5$. The learning rate linearly increases from 0 to $5e-4$ during the first 3000 steps while linearly decreasing to 0 in the last 47000 steps, which is 1/10 comparing to DNABERT2. We use mask modeling as training strategy.

**For neural network architecture predictor,** we use Adam optimizer with learning rate = 1e-3, weight decay = 1e-5, batch size = 32, num epochs = 50, dropout rate = 0.3. We design two settings by if there is very similar tasks in the training set and test set. For example, Transcription Factor Prediction 0-4 are considered as similar tasks, Subcellular Location Prediction and Binary Location Prediction are considered as similar tasks as well. For supervise setting, we use tasks $[0, 1, 4, 5, 7, 8, 9, 11, 12, 13, 14, 16]$ as training set and tasks $[2, 3, 6, 10, 15, 17]$ as test set. There are 6446 data points in the training set and 3210 data points in test set. For transfer setting, we use tasks $[5, 6, 7, 8, 9, 10, 12, 13, 14]$ as training set and tasks $[2, 11, 16, 17]$ as test set. There are 4793 data points in the training set and 2156 data points in test set. We use all-MiniLM-L6-v2 to encode task descriptions in to task embeddings. We set the prediction number K to be 3.

**For LLM and Agent system architecture predictor**, the default LLM we used is GPT-4o. We also conduct ablation study on the model choices. We set the prediction number K to be 3.

**For task-specific training**, all the hyperparameters are listed in the follow Table 4.

### A.5.2. EVALUATION SETTING DETAILS

In this section, we clarify the specific evaluation protocols corresponding to the model variants reported in Table 1 and Table 2. We employed two distinct weight initialization paradigms as outlined in Section 3.3 including *Training from Scratch* and *Pretrain and Finetune*.

**Pretrain and Finetune.** The BIOARC variants involve initializing the architecture with weights inherited from a supernet that has been pretrained on large-scale biological data, followed by task-specific fine-tuning. These variants are distinguished by the self-supervised learning objective used during the supernet pretraining phase. Specifically, **BIOARC (mask-ft)** utilizes weights from a supernet pretrained via Masked Modeling; **BIOARC (con-ft)** employs weights from a supernet pretrained using Contrastive Learning; and **BIOARC (ntp-ft)** initializes from a supernet trained with the Next Token Prediction objective.

### A.5.3. COMPUTATIONAL COST

We report the computational costs for both supernet pretraining and task-specific finetuning. All experiments were conducted on NVIDIA A100 80G GPUs. For the **supernet pretraining**, the computational cost varies by objective. Specifically, mask modeling requires approximately 1.9 hours per epoch, while contrastive learning takes roughly 4.1 hours per epoch. **Training the foundation model** via mask modeling took a total of 10.9 hours. For **task-specific training**, we evaluate the total time required to finetune all 360 architectures across 18 downstream tasks, shown in Table 5.

*Table 5.* Computational cost for task-specific finetuning across 18 tasks.

| **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|
| 1h 40m | 1h 30m | 57m | 1h 20m | 58m | 2h 30m | 2h 13m | 14m | 5h 50m |
| **9** | **10** | **11** | **12** | **13** | **14** | **15** | **16** | **17** |
| 5h 12m | 36m | 5h 56m | 6h 43m | 72h 20m | 7h 25m | 85h 15m | 6h 02m | 9h 51m |

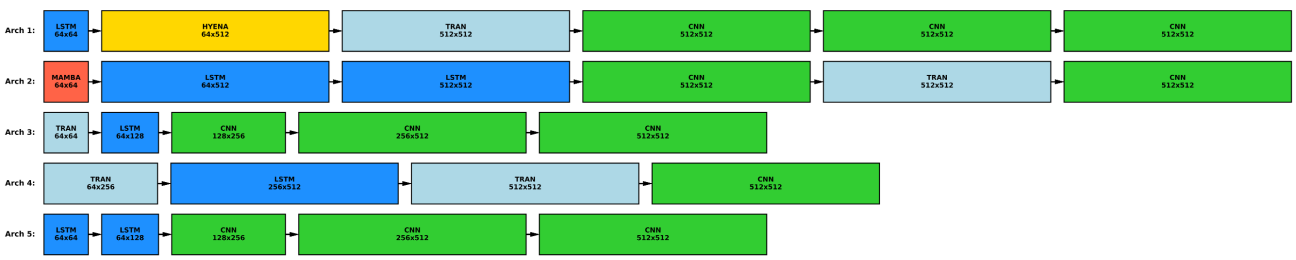



*Figure 7.* This figure illustrates the top five performing protein model architectures (Arch 1-5).

## A.6. Main experiment results

### A.6.1. Top Architectures Pattern

Similar to DNA, top architectures for protein demonstrate an observable pattern with LSTMs and Transformers in the initial stages followed by CNNs, as shown in Figure 7. Moreover, this pattern for protein is not consistent with the pattern for DNA, validating our intuition that architecture should be customized for different modalities.

### A.6.2. Architecture Similarity

To illustrate our encoding scheme, consider a sequential architecture consisting of three layers: a CNN layer (64 to 128 channels), followed by a Transformer block (128 to 256 dim), and an LSTM layer (256 to 512 dim).

**Adjacency Matrix.** We represent the connectivity between these $N = 3$ layers using an adjacency matrix $\mathbf{A} \in \{0, 1\}^{N \times N}$. For a strictly sequential architecture, this forms a super-diagonal matrix:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

**Node Feature Matrix.** The features for each layer are encoded in $\mathbf{X} \in \mathbb{R}^{N \times F}$. Each row corresponds to a layer, concatenating the one-hot encoded operation type (first 5 columns) and the normalized input/output dimensions (last 2 columns):

$$\mathbf{X} = \begin{array}{c} \\ \text{CNN} \\ \text{Trans} \\ \text{LSTM} \end{array} \begin{array}{c} \begin{array}{ccccc} \multicolumn{5}{c}{\text{Type}} & \text{In} & \text{Out} \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0.14 \\ 0 & 0 & 1 & 0 & 0 & 0.14 & 0.43 \\ 0 & 0 & 0 & 1 & 0 & 0.43 & 1.00 \end{pmatrix} \end{array}$$

Dimensions are normalized by a factor of $D_{\text{max}} = 512$. The rows correspond to [CNN, 64, 128], [Transformer, 128, 256], and [LSTM, 256, 512], respectively.

We compare the average embedding of top 10% performing architectures of each task. Each architecture in encoded with one-hot embedding. As is shown in Figure 8, similar tasks tend to have similar architecture embeddings.

### A.6.3. More results on scaling up

To further investigate the impact of architecture depth and the scalability of our searched architecture, we expanded discovered optimal architecture (top 1 in Figure 2) into a 74.7M-parameter foundation model by stacking 2 Hyena, 4 Transformer, and 3 CNN layers with a hidden dimension of 1024. We extend the training steps to 100,000 and keep other hyperparameters the same. As shown in Table 6, this scaling yields a improvement on 10/12 tasks, which validates the architecture's effectiveness for large-scale pre-training. **This demonstrates that our discovered architecture effectively supports scaling.**

### A.6.4. Extended DNA evaluation against recent genomic foundation models

To position BioArc-F against the most recent genomic foundation models, we additionally evaluate it on the Nucleotide Transformer (NT) benchmark and the Genomic Benchmarks (GB), and compare against Caduceus-Ph, Caduceus-PS (Schiff

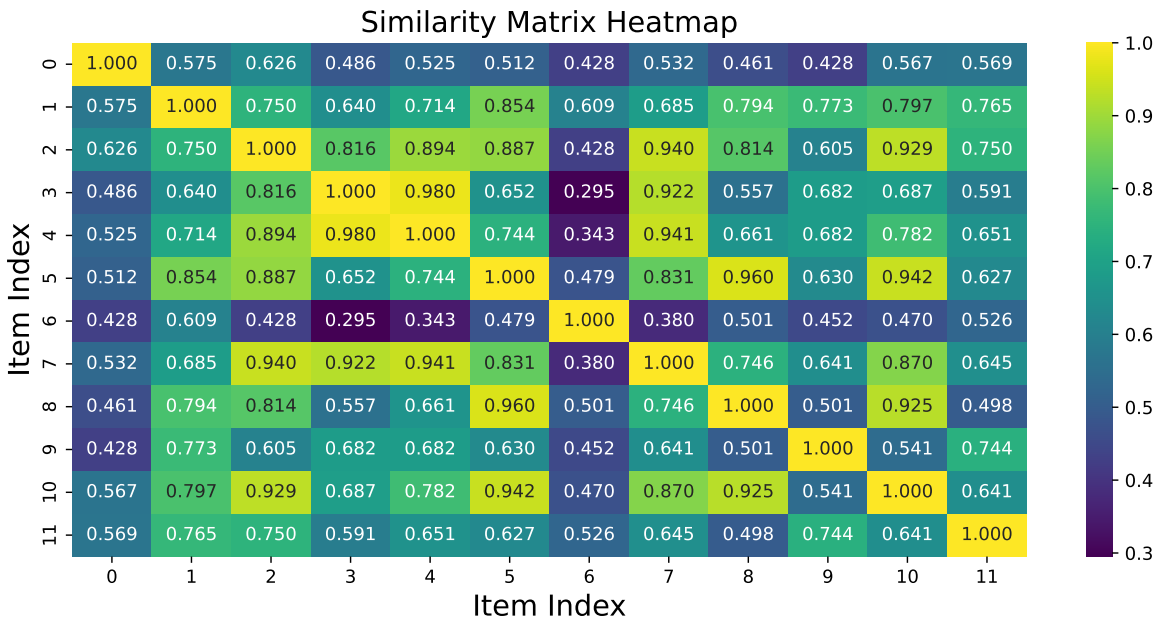



*Figure 8.* The heatmap showing the cosine similarity between different neural network architectures. The color and value in each cell represent the similarity score between two architectures, with values closer to 1.0 (yellow) indicating greater similarity.

*Table 6.* Comparison of performance across 12 DNA tasks (Tasks 0-11) between BioArc identified foundation models and state-of-art baseline.

| Model | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| VQDNA (HRQ) | 72.48 | 76.43 | 66.85 | 58.92 | 78.10 | 71.02 | 70.58 | 78.50 | 90.75 | 94.48 | 74.52 | **89.53** |
| BioArc-FM-4.8M | 82.50 | 85.10 | 76.40 | 72.40 | 82.20 | 80.47 | 83.34 | 76.35 | 90.24 | 95.25 | 79.93 | 68.26 |
| BioArc-FM-88M | **84.30** | 83.60 | 85.70 | **78.80** | 88.60 | 83.09 | 82.70 | 86.62 | 92.28 | 96.01 | 79.61 | 84.83 |

et al., 2024), and GENERator (Wu et al., 2025). Baseline numbers are taken from the GENERator paper (its Tables S4 and S5), and BIOARC-F is evaluated under the same protocol. As shown in Table 7 (NT, MCC) and Table 8 (GB, accuracy), **despite being far smaller than GENERator**, BIOARC-F is highly competitive: on the NT benchmark it achieves the best result on all histone-mark and enhancer tasks (with a large margin on enhancer detection, $0.874$ vs. $\leq 0.580$), while GENERator's much larger generative pretraining gives it the edge on promoter and splice-site tasks. On the Genomic Benchmarks, BIOARC-F leads on several human enhancer/promoter datasets (e.g., Human nonTATA Promoter, Human Enhancer Ensembl) and is within 1–2 points of the best model elsewhere.

### A.6.5. MORE RESULTS ON PROTEIN

We provide additional protein results in this section. The main downstream comparison, including the controlled experiment in which BIOARC 8M and a reimplemented ESM-2 8M are pretrained under identical conditions (full UniRef50, 50K steps) to isolate architecture, is reported in Table 2 in the main text. Here we additionally report the corresponding pretraining loss in Table 9: under the matched budget, BIOARC 8M attains lower masked-language-modeling loss than ESM-2 8M at every checkpoint and is still decreasing at 50K steps, corroborating that its downstream advantage stems from greater data efficiency rather than a larger pretraining budget.

### A.6.6. MORE RESULTS ON DIFFERENT TRAINING STRATEGIES

The effect of different training strategies is shown in Figure 9. It indicates that pretraining provides no clear advantage, and the choice of pretraining strategy appears to have minimal effect on the outcome.

*Table 7.* NT benchmark results (MCC). Baselines are from Table S4 of GENERator (Wu et al., 2025); BIOARC-F is evaluated under the same protocol. **Bold** marks the best result per task.

| Task | BIOARC | Caduceus-Ph | Caduceus-PS | GENERator |
|---|---|---|---|---|
| H3 | 0.783 | 0.794 | 0.772 | **0.806** |
| H3K14ac | **0.648** | 0.564 | 0.596 | 0.605 |
| H3K36me3 | 0.647 | 0.590 | 0.611 | **0.657** |
| H3K4me1 | **0.571** | 0.468 | 0.487 | 0.553 |
| H3K4me2 | **0.547** | 0.332 | 0.431 | 0.424 |
| H3K4me3 | **0.640** | 0.490 | 0.528 | 0.512 |
| H3K79me3 | **0.738** | 0.641 | 0.682 | 0.670 |
| H3K9ac | **0.644** | 0.575 | 0.564 | 0.612 |
| H4 | 0.809 | 0.788 | 0.799 | **0.815** |
| H4ac | **0.685** | 0.548 | 0.585 | 0.592 |
| Enhancers | **0.874** | 0.522 | 0.511 | 0.580 |
| Enhancers types | **0.782** | 0.403 | 0.410 | 0.477 |
| Promoter all | 0.941 | 0.937 | 0.941 | **0.962** |
| Promoter no TATA | 0.940 | 0.935 | 0.940 | **0.962** |
| Promoter TATA | 0.924 | 0.895 | 0.903 | **0.948** |
| Splice sites all | 0.971 | 0.935 | 0.953 | **0.978** |
| Splice acceptors | 0.953 | 0.918 | 0.907 | **0.981** |
| Splice donors | 0.963 | 0.912 | 0.930 | **0.978** |

*Table 8.* Genomic Benchmarks results (accuracy, %). Baselines are from Table S5 of GENERator (Wu et al., 2025); BIOARC-F is evaluated under the same protocol. **Bold** marks the best result per task.

| Task | BIOARC | Caduceus-Ph | Caduceus-PS | GENERator |
|---|---|---|---|---|
| Coding vs. Interg | 0.949 | 0.933 | 0.944 | **0.963** |
| Drosophila Enh | 0.819 | **0.827** | 0.816 | 0.821 |
| Human Enh Cohn | 0.750 | 0.747 | 0.749 | **0.763** |
| Human Enh Ensembl | **0.934** | 0.924 | 0.923 | 0.917 |
| Human Ensembl Reg | **0.941** | 0.938 | **0.941** | 0.928 |
| Human nonTATA Prom | **0.975** | 0.961 | 0.961 | 0.958 |
| Human OCR Ensembl | 0.820 | 0.825 | **0.826** | 0.823 |
| Human vs. Worm | 0.972 | 0.975 | 0.976 | **0.980** |
| Mouse Enh Ensembl | 0.820 | 0.807 | 0.813 | **0.871** |

*Table 9.* Pretraining masked-language-modeling loss (↓) under the controlled setting (full UniRef50, 50K steps, identical hyperparameters). BIOARC 8M achieves lower loss than ESM-2 8M at every checkpoint and is still decreasing at 50K steps.

| Model | 1K | 5K | 10K | 20K | 30K | 40K | 50K |
|---|---|---|---|---|---|---|---|
| BIOARC 8M | **2.783** | **2.690** | **2.647** | **2.617** | **2.604** | **2.593** | **2.585** |
| ESM-2 8M | 2.906 | 2.749 | 2.701 | 2.656 | 2.635 | 2.621 | 2.607 |

### A.6.7. MORE RESULTS ON DIFFERENT TOKENIZERS

Here we show more results of the effect of different tokenizers on different architectures, shown in Figure 10. Our results highlight two key findings regarding tokenization. First, different architectures exhibit distinct preferences: Mamba works best with 1-mer, while Transformer favors 6-mer. Second, these preferences are task-dependent; the LSTM architecture, for example, excels with 6-mer tokenization on tasks 0–4 but requires 1-mer tokenization for optimal performance on tasks 5–10.

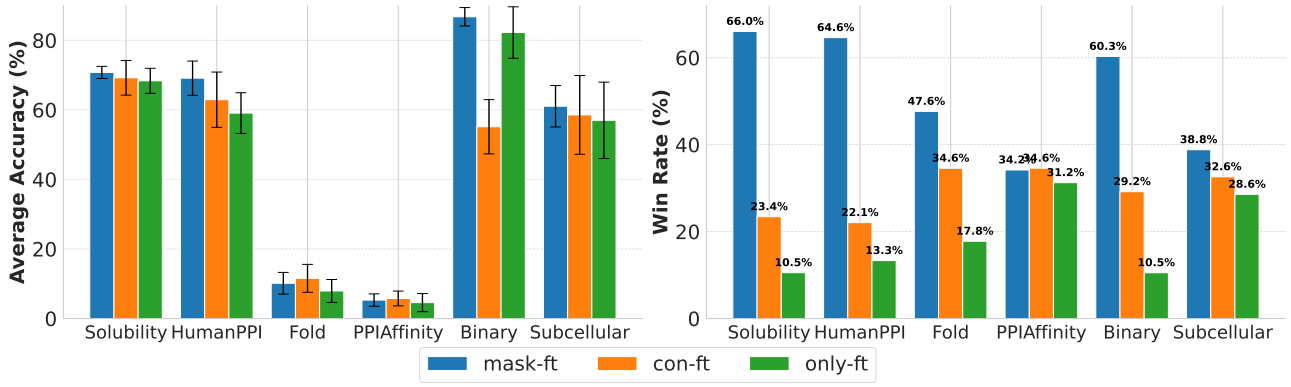
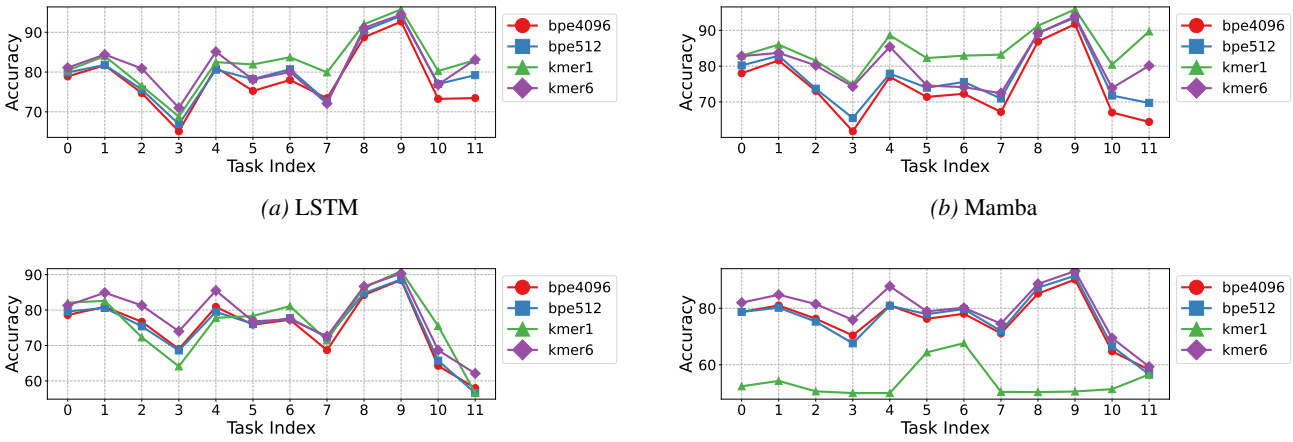



*Figure 9.* Performance of different training strategies on Protein. In the left panel, each cell shows the mean performance ± standard deviation across all architectures. In the right panel, each bar shows the percentage of the total 360 architectures that chose that training strategy to yield the best performance.




*(a)* LSTM *(b)* Mamba

*(c)* Hyena *(d)* Transformer

*Figure 10.* Performance of different tokenization methods on various architectures.

## A.7. More Analysis

### A.7.1. HYBRID AND SINGLE-MODULE ARCHITECTURE COMPARISON

We compare the performance of BIOARC hybrid architectures against optimal single-module architectures. To ensure a fair comparison, the single-module baselines were not arbitrarily selected. They represent the top-performing candidates discovered by restricting the search space to exclusively use a specific module type(e.g., an all-CNN or all-Transformer search space). All models, including both hybrid and single-module variants, were trained from scratch to eliminate weight-sharing bias. As shown in Figure 11, **hybrid architectures consistently outperform the best single-module baselines,** which validates our intuition in designing a search space that allows for the combination of different architectures.

### A.7.2. LAYER-WISE CONTRIBUTION ANALYSIS

To verify that BIOARC superiority stems from the synergistic composition of hybrid modules rather than a single dominant layer, we conducted a layer-wise contribution analysis on the top-performing architecture (rank-1 in Figure 2). We trained a separate classification head on the output of each layer across all DNA tasks to evaluate intermediate feature quality. As shown in Table 10, we observe a consistent monotonic improvement in average accuracy (from 70.32% to 83.18%) alongside a progressive reduction in standard error. This steady gain confirms that the specific sequence of diverse modules collaboratively refines biological representations, validating that the model's effectiveness relies on the holistic hybrid design.


*Figure 11.* Performance of hybrid architecture compared with optimal single-module architectures found via restricted search.

*Table 10.* Average accuracy and standard error across different layers.

| Metric | Layer 0 | Layer 1 | Layer 2 | Layer 3 | Layer 4 | Layer 5 |
|---|---|---|---|---|---|---|
| Accuracy (%) | $70.32 \pm 0.89$ | $74.07 \pm 0.86$ | $75.09 \pm 0.82$ | $78.36 \pm 0.62$ | $82.09 \pm 0.49$ | $83.18 \pm 0.46$ |

*Table 11.* Performance statistics of architectures grouped by depth. Metrics (Mean, Std, Min, Max, Median) are calculated based on the **average accuracy across all tasks**. All results are obtained under **fixed hyperparameters**.

| Depth | Count | Mean (%) | Std | Min (%) | Max (%) | Median (%) |
|---|---|---|---|---|---|---|
| 3 Layers | 60 | 81.00 | 4.23 | **53.77** | 84.85 | 81.49 |
| 4 Layers | 100 | **81.74** | 5.57 | 53.38 | 85.14 | 83.07 |
| 5 Layers | 100 | 77.74 | 10.24 | 47.44 | 85.48 | 82.56 |
| 6 Layers | 100 | 78.17 | 10.72 | 47.70 | **86.53** | **83.30** |

### A.7.3. ARCHITECTURE DEPTH ANALYSIS

We investigate the relationship between network depth and transfer performance by analyzing the statistics of architectures ranging from 3 to 6 layers. For each of the sampled architectures, we compute the **average accuracy across all 12 DNA downstream tasks** to represent its overall generalization capability. From Table 11, we observe that while deeper architectures demonstrate higher median and maximum performance, they also exhibit lower mean accuracy and higher standard deviations. This suggests that deeper models are more sensitive/susceptible to the fixed hyperparameter settings. **This implies that depth is not the sole determinant of performance; rather, the structural composition of the architecture is key.** We also observe that in top architectures of 4 and 5 layers demonstrate structure as 6 layers in Figure 12. **This indicates that once an effective architectural pattern is identified, scaling up the depth can further enhance performance.**

### A.7.4. PERFORMANCE-PARAMETER ANALYSIS

To investigate whether the performance of BIOARC-discovered models stems from increased capacity or superior design, we analyzed the relationship between parameter count and average accuracy across the search space as shown in Figure 13. We evaluated all paths directly from the pretrained supernet by freezing the backbone weights and training only a linear classification head. We observe no positive correlation between model size and performance. High accuracy scores are achieved by efficient architectures (yellow points) without relying on large parameter budgets, while increasing model size does not guarantee performance gains. The substantial performance variance observed among models of identical size

indicates that the specific topological arrangement of modules—the inductive bias—is the decisive factor for biological sequence modeling. This confirms that BIOARC's effectiveness lies in discovering these optimal architectural patterns rather than simple parameter scaling.

### A.7.5. ARCHITECTURE RANKING CORRELATION ANALYSIS

To validate our evaluation protocol and justify the necessity of the decoupled pretrain-then-finetune strategy, we conducted a two-fold correlation analysis. We compare the ranking consistency across three distinct experimental settings for the 360 representative candidate architectures:

- **Training from Scratch Individually (Ground Truth):** Architectures are initialized randomly and trained independently on downstream tasks. This serves as the performance baseline.
- **Supernet Pretraining + Supernet Finetuning:** Architectures are evaluated directly using the shared weights from the supernet (after pretraining and supernet-level finetuning), without decoupling the parameters for each path.
- **Supernet Pretraining+ Individual Finetune (Our Method):** Architectures inherit weights from the pretrained BIOARC supernet but are finetuned independently.

**Necessity of Individual Finetuning.** First, we investigate whether we could directly finetune the supernet for correct ranking of architectures. We observe a low correlation between ***Supernet Pretraining + Supernet Finetuning*** and the ***Ground Truth***, as shown in Figure 14. This discrepancy indicates the phenomenon of weight interference and co-adaptation prevents the shared parameters from accurately reflecting the distinct potential of specific paths. Consequently, this validates our methodological choice in Section 3.3 that we cannot rely on the supernet for direct ranking. Instead, evaluating architectures requires **individual finetuning** to decouple weights and reveal the true performance potential of each candidate.

**Validation of Pretraining Effectiveness.** Second, we assess the correlation between ***Supernet Pretraining+ Individual Finetune*** and ***Training from Scratch***. As illustrated in Figure 15, we observe a strong positive correlation ($\rho = 0.7287$) between these two protocols. This high consistency demonstrates two critical points: (1) The performance ranking of architectures is intrinsic to their topology and remains robust regardless of the initialization strategy (random vs. pretrained). (2) The BIOARC supernet successfully learns high-quality, transferable features during pretraining, as it preserves the ground-truth ranking order when used as an initialization backbone.

### A.7.6. SHARED-BLOCK GRADIENT STABILITY

A potential concern with weight-sharing supernets is that a block shared across many sampled paths receives gradients from stochastically varying predecessors and successors, which could destabilize its optimization. We address this directly by tracking, for each shared block, the **standard deviation of its gradient norm across all paths that contain it**, and showing that this variance decreases substantially and stabilizes over training. We report results on (1) a controlled supernet and (2) our original supernet, and note that this stability is consistent with established theoretical and empirical findings in the weight-sharing NAS literature (Li et al., 2021).

**Controlled supernet.** We construct a 3-layer supernet with 5 block types per layer ($5^3 = 125$ paths, 15 shared blocks). We focus on middle-layer blocks (Layer 1, $64 \rightarrow 128$), as each block's 25 containing paths exhaust all predecessor $\times$ successor type combinations ($5 \times 5$), providing the strictest test of robustness to upstream and downstream distributional shifts. Gradient norms are computed on the same fixed mini-batches. As shown in Table 12, the gradient-norm standard deviation of every shared block decreases substantially from initialization and stabilizes over training, confirming that each block converges stably despite receiving inputs from stochastically sampled predecessors.

**Original supernet.** To confirm these findings are not an artifact of the simplified setup, we repeat the analysis on our original supernet. We select 5 representative shared blocks, one per block type, spanning different dimensions and layer positions, and track their gradient-norm standard deviation across all containing paths (gradient norms computed on the same fixed mini-batches). As shown in Table 13, all five blocks exhibit an overall decaying trend, stabilizing after the initial training phase. Crucially, the *mean* gradient norm remains on a healthy scale throughout (e.g., CNN: $0.115 \rightarrow 0.099$,

*Table 12.* Gradient-norm standard deviation of each middle-layer shared block (Layer 1, $64 \to 128$) in the controlled supernet, computed across all 25 containing paths over training epochs (E$x$).

| Block | Init | E1 | E2 | E5 | E10 | E15 | E20 |
|---|---|---|---|---|---|---|---|
| CNN | 0.769 | 0.150 | 0.087 | 0.048 | 0.037 | 0.030 | 0.036 |
| Hyena | 8.719 | 0.196 | 0.175 | 0.131 | 0.123 | 0.123 | 0.141 |
| LSTM | 14.297 | 0.331 | 0.187 | 0.175 | 0.203 | 0.201 | 0.210 |
| Mamba | 3.238 | 0.196 | 0.159 | 0.129 | 0.118 | 0.124 | 0.132 |
| Transformer | 3.627 | 0.605 | 0.170 | 0.115 | 0.107 | 0.115 | 0.124 |

Transformer: $0.247 \to 0.287$ from E4 to E10), confirming that the standard-deviation decay reflects genuine convergence of block representations across paths rather than gradient vanishing.

*Table 13.* Gradient-norm standard deviation of 5 representative shared blocks in the original supernet, tracked across all containing paths over training epochs (E$x$).

| Block | Init | E1 | E2 | E4 | E6 | E8 | E10 |
|---|---|---|---|---|---|---|---|
| CNN ($64 \to 128$) | 0.507 | 0.054 | 0.126 | 0.044 | 0.038 | 0.047 | 0.043 |
| Transformer ($512 \to 512$) | 1.905 | 0.341 | 0.795 | 0.172 | 0.145 | 0.162 | 0.170 |
| Hyena ($512 \to 512$) | 4.898 | 1.568 | 2.537 | 0.129 | 0.100 | 0.097 | 0.098 |
| Mamba ($64 \to 128$) | 0.783 | 0.129 | 0.273 | 0.069 | 0.056 | 0.060 | 0.071 |
| LSTM ($256 \to 256$) | 3.298 | 0.218 | 0.452 | 0.113 | 0.103 | 0.113 | 0.126 |

### A.7.7. ARCHITECTURE PREDICTION

Training every candidate in a vast BIOARC search space is computationally expensive. Leveraging our observation that **optimal architectures for functionally similar tasks exhibit significant topological similarity**, we propose a hierarchy of three approaches, ranging from numerical regression to high-level semantic reasoning agents, to efficiently exploit BIOARC explored (architecture, task, performance) tuples for identifying optimal architectures for unseen biological tasks.

**Experiment Settings** Based on our observations from RQ2, we conducted experiments in both supervised and transfer settings with detailed data split available in Appendix A.5.1. Unlike standard classification, the "optimal" architecture is a ranking relative to the search space. We define our metrics as follows:

- **Ground Truth Construction ($\mathcal{G}_t$):** For every task $t$ in our benchmark, we utilize the exhaustive evaluation results from Section 3.4. We define the Ground Truth Set $\mathcal{G}_t$ as the top architectures ranked by their performance for each task. These represent the empirically verified optimal designs.
- **Metric Definitions:** Let $\mathcal{P}_t^k$ be the set of top-$k$ architectures predicted by our model for task $t$. We evaluate the alignment between prediction and reality using:
  1. **Precision@k ($|\mathcal{P}_t^k \cap \mathcal{G}_t|/k$):** This measures the *efficiency* of the prediction. It quantifies the proportion of the suggested architectures that are truly top-tier, indicating the reliability of the predictor's advice to a user with a limited budget for training trials.
  2. **Recall@k ($|\mathcal{P}_t^k \cap \mathcal{G}_t|/|\mathcal{G}_t|$):** This measures the *coverage* of the design space. It assesses the predictor's ability to uncover the diverse set of high-performing candidates, preventing mode collapse into a single architecture type.
  3. **Hit Rate@k ($\mathbb{I}(|\mathcal{P}_t^k \cap \mathcal{G}_t| \geq 1)$):** This measures the *probability of success*. It indicates whether the user will find *at least one* optimal architecture within the top-$k$ predictions, serving as a critical "success/failure" metric for practical deployment.

**Neural Network Prediction** We design a neural predictor denoted as $\mathcal{P}$ that maps an architecture-task pair $(a, t)$ to a predicted performance score. The model takes an architecture embedding $\mathbf{h}_a$ and a task embedding $\mathbf{h}_t$ as input and is trained

*Table 14.* Functional breakdown of the BIOARC Agent pipeline. The system progresses from parsing raw text to synthesizing an optimal design.

| Role | Input | Output |
|---|---|---|
| **Analyst** | Raw user task description | Structured metadata (e.g., modality, objective) |
| **Task Retriever** | Structured metadata | Semantically aligned tasks in Knowledge Base |
| **Arch. Retriever** | Aligned tasks | Proven architectures & empirical performance |
| **Predictor** | Retrieved architectures & metrics | Predict optimal architecture design |

by minimizing the Mean Squared Error against the ground-truth performance $y_{a,t}$. This objective is formally expressed through the loss function $\mathcal{L}(\mathcal{P})$:

$$\mathcal{L}(\mathcal{P}) = \mathbb{E}_{(a,t)\sim\mathcal{A}}\left[\left(\mathcal{P}(\mathbf{h}_a, \mathbf{h}_t) - y_{a,t}\right)^2\right] \tag{10}$$

Following previous work (Ma et al., 2019; White et al., 2021), we represent each architecture $a$ as a graph with its node features $\mathbf{h}_a$ formed by concatenating its one-hot encoded module type $\mathrm{O}(m_i)$ and the normalized dimensions of its input and output, $Z(h_{i-1})$ and $Z(h_i)$. For task embeddings, a pretrained language model (PLM) is used to encode the task's textual description, $d_{\text{task}}$, into a vector $\mathbf{h}_t$. These encoding processes are defined as:

$$\mathbf{X}_a = (\mathrm{O}(m_i) \| Z(h_{i-1}) \| Z(h_i))_{i=1}^{d}, \qquad \mathbf{h}_a = \mathbf{GNN}(\mathbf{I}, \mathbf{X}_a), \qquad \mathbf{h}_t = \mathrm{PLM}(d_{\text{task}}) \tag{11}$$

where $d$ is the depth of the architecture. An example of architecture embedding is in Appendix A.6.2.

**LLM + RAG** Moving beyond numerical regression, we leverage LLMs augmented with retrieval capabilities. To provide relevant context, we first encode the input task description into **an embedding vector and retrieve the top-$n$ most similar historical tasks** based on vector similarity. These retrieved tasks, along with their corresponding top-$k$ architectures, are formatted as a **knowledge base** and **performance records** within the prompt. The LLM is then instructed to act as an analyst: it first evaluates the new task's characteristics, identifies the most relevant matches from the retrieved knowledge base, and predicts the top-$m$ architecture. This approach enables the model to explicitly reason about task relationships and empirical performance. $n, k, m$ are hyperparameters. Detailed LLM prompts are provided in Appendix A.9.

**BIOARC Agent** To address the limitations of simple embedding-based retrieval in capturing biological nuances, we introduce BIOARC Agent. As detailed in Table 14, this system transforms unstructured queries into empirically grounded predictions. Specifically, the system identifies the top-$k$ most semantically similar historical tasks and retrieves their corresponding top-$n$ high-performing architectures. Then Predictor conducts reasoning and output top-$m$ architectures. $n, k, m$ are hyperparameters. Full prompts and operational details are provided in Appendix A.10.

**Experiment Results** We use GPT-4o as default backbone and compared the performance of multiple LLM models. From Table 15, **we find that BIOARC Agent consistently outperforms other methods in both settings.** We attribute this improvement to the modular multi-agent design, which decouples the decision-making process to minimize hallucination and error propagation. Furthermore, unlike static embeddings, our natural language-driven retrieval ensures a semantic matching of tasks, allowing the system to identify historical precedents that share genuine structural and functional requirements.

We benchmarked several LLMs across different scales, ranging from small open-source models (Qwen3-4B, Llama-3.1-8B) to proprietary frontier models (GPT-4o, GPT-5), reported in Table 16. Smaller models, such as Qwen3-4B (both Instruct and Thinking variants), failed to produce valid predictions (yielding 0.00 scores), while Llama-3.1-8B showed only marginal performance. This suggests that the complexity of the BIOARC architecture search space exceeds **the reasoning capacity of current small-scale models.** The Supervised setting, as expected, generally yields higher precision and recall than the Transfer setting. **This aligns with our observation that similar downstream tasks tend to benefit from similar architectures.**

### A.8. Hybrid Architecture Interpretation Analysis

We validate our hybrid architecture (Hyena-Transformer-CNN) by analyzing whether it captures the **biological grammar** of human genomic sequences, focusing specifically on the challenging task of **Core Promoter Detection (no-TATA)**.

*Table 15.* Architecture prediction methods comparison under supervised and transfer setting.

| Setting | Metric | NN Predictor | | | | | LLM+RAG | | | BIOARC Agent | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | @5 | @10 | @15 | @20 | @30 | @1 | @3 | @5 | @1 | @3 | @5 |
| **Supervised** | Hit Rate | 0.167 | 0.333 | 0.333 | 0.333 | 0.667 | 0.000 | 0.167 | 0.167 | **0.500** | **0.500** | **0.500** |
| | Precision | 0.033 | 0.033 | 0.022 | 0.017 | 0.028 | 0.000 | 0.056 | 0.111 | **0.500** | **0.444** | **0.300** |
| | Recall | 0.033 | 0.067 | 0.067 | 0.100 | 0.167 | 0.000 | 0.056 | 0.067 | **0.100** | **0.267** | **0.300** |
| **Transfer** | Hit Rate | 0.000 | 0.250 | 0.250 | 0.250 | 0.500 | 0.000 | 0.000 | 0.000 | **0.250** | **0.250** | **0.500** |
| | Precision | 0.000 | 0.025 | 0.017 | 0.013 | 0.017 | 0.000 | 0.000 | 0.000 | **0.250** | **0.250** | **0.300** |
| | Recall | 0.000 | 0.050 | 0.050 | 0.050 | 0.100 | 0.000 | 0.000 | 0.000 | **0.050** | **0.150** | **0.300** |

*Table 16.* Architecture prediction results with our Agent system.

| Setting | Model | Precision | | | Hit Rate | | | Recall | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | @1 | @3 | @5 | @1 | @3 | @5 | @1 | @3 | @5 |
| **Supervised** | GPT-5 | 0.33 | 0.28 | 0.17 | 0.33 | 0.33 | 0.33 | 0.07 | 0.17 | 0.17 |
| | GPT-4o | 0.50 | 0.44 | 0.30 | 0.50 | 0.50 | 0.50 | 0.10 | 0.27 | 0.30 |
| | Llama-3.1-8B | 0.17 | 0.06 | 0.03 | 0.17 | 0.17 | 0.17 | 0.03 | 0.03 | 0.03 |
| | Qwen3-4B-Instruct | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 |
| | Qwen3-4B-Thinking | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 |
| **Transfer** | GPT-5 | 0.25 | 0.25 | 0.25 | 0.25 | 0.25 | 0.25 | 0.05 | 0.15 | 0.25 |
| | GPT-4o | 0.25 | 0.25 | 0.30 | 0.25 | 0.25 | 0.50 | 0.05 | 0.15 | 0.30 |
| | Llama-3.1-8B | 0.25 | 0.08 | 0.05 | 0.25 | 0.25 | 0.25 | 0.05 | 0.05 | 0.05 |
| | Qwen3-4B-Instruct | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 |
| | Qwen3-4B-Thinking | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 |

### A.8.1. TASK AND GRAMMAR DESCRIPTION

The model analyzes a short 70bp DNA window ($-35$bp to $+35$bp) centered on the Transcription Start Site (TSS). Unlike TATA promoters which contain strong, identifiable motifs, **no-TATA promoters** (which constitute $\sim 76\%$ of human genes) rely on weaker, combinatorial signals. The model must implicitly learn a grammar distinct from the classical TATA paradigm:

**Core Elements (Vocabulary)** In the absence of the TATA-box, the recognition vocabulary expands to a cooperative set of motifs. We focus on two non-canonical categories: (1) **DPE-driven Promoters** (Burke & Kadonaga, 1997), which rely on the synergy between the Initiator (Inr) and downstream elements like the MTE and **DPE** (see Table 17); and (2) **CpG-island Promoters** (Deaton & Bird, 2011), characterized by high GC-content and a lack of specific motifs.

**Positional Constraints (Syntax)** For no-TATA focused promoters, the spatial syntax is rigid to compensate for weaker binding affinity:

- **Inr-DPE Spacing:** The DPE functions strictly when located exactly $+28$ to $+32$bp downstream of the Inr (Kutach & Kadonaga, 2000).
- **TSS Definition:** Without a TATA box ($\sim -30$bp), the Inr element itself becomes the primary determinant of the $+1$ position.

**Combinatorial Rules (Semantics)** The model must distinguish "functional grammar" from random occurrences. A critical semantic rule is the **Inr+DPE synergy**: neither element strongly recruits transcription machinery alone, but their simultaneous presence at the correct distance creates a high-affinity binding site. Alternatively, in CpG contexts, the model detects "broad" initiation patterns rather than precise motifs.

*Table 17.* Key Elements in no-TATA Human Core Promoters. Since the TATA-box is absent, recognition relies on the Initiator and downstream regions. Positions are relative to the TSS (+1).

| Element | Position | Abbr. | Consensus Sequence |
|---|---|---|---|
| Initiator | $-2$ to $+5$ | Inr | YYANWYY (Javahery et al., 1994) |
| Motif Ten Element | $+18$ to $+27$ | MTE | CSARCSSAACGS |
| Downstream Element | $+28$ to $+32$ | DPE | RGWYV |

*Note:* **R**=Purine, **Y**=Pyrimidine, **W**=Weak (A/T), **S**=Strong (G/C), **N**=Any. **These canonical consensus sequences are defined primarily in *Drosophila* and are often more degenerate in humans.**

### A.8.2. INTERPRETATION METHODOLOGY

To decode how distinct layers process biological information, we employ layer-specific visualization techniques tailored to the mathematical operations of each module.

**Hyena: Norm-based Activation Profiling** Since Hyena operators process long-range context via implicit convolutions, individual dimensions do not necessarily correspond to discrete features. Instead, we measure the **information density** at each position $t$ by computing the $L_2$ norm of the hidden state vector $h_t \in \mathbb{R}^d$:

$$A_{\text{Hyena}}(t) = ||h_t||_2 = \sqrt{\sum_{i=1}^{d} (h_{t,i})^2} \tag{12}$$

Peaks in $A_{\text{Hyena}}(t)$ indicate regions where the model aggregates significant global context, highlighting semantically rich sequence segments.

**Transformer: Attention Accumulation** To understand syntactic relationships, we analyze the self-attention matrix $\alpha$. Rather than examining pairwise weights in isolation, we calculate the **Total Attention Received** for each token $j$ by summing the attention weights from all query positions $i$:

$$S_{\text{Attn}}(j) = \sum_{i=1}^{L} \alpha_{i,j} \tag{13}$$

Tokens with high $S_{\text{Attn}}(j)$ act as "Attention Magnets," representing structural pivots. **For visualization, $S_{\text{Attn}}$ is min-max normalized to the range $[0, 1]$ to highlight relative structural importance independent of absolute magnitude.**

**CNN: Importance Ranking and Maximal Activation** Given the large number of convolutional kernels, we first prioritize them based on their contribution to the model's decision. We compute the **gradient-weighted importance** for each kernel $k$ (following the Grad-CAM protocol) and select the top-$K$ filters with the highest total contribution. For these top-ranked filters, we identify the specific input sequence window $x_{[\hat{t}:\hat{t}+w]}$ that triggers the maximum activation:

$$\hat{t} = \underset{t}{\text{argmax}} \, |(W_k * X)_t| \tag{14}$$

By visualizing the subsequence at position $\hat{t}$, we reconstruct the specific DNA vocabulary (motifs) that the network deems most critical for prediction.

### A.8.3. VALIDATION OF GRAMMAR CAPTURE: A LAYER-WISE ANALYSIS

To interpret how the hybrid architecture identifies no-TATA promoters, we visualize the activation patterns across different layers (Figure 4). The analysis reveals a hierarchical processing strategy that transitions from global sequence context to localized motif integration.

**Hyena (Layer 0): Global Contextual Embedding.** The Hyena layer acts as a global semantic encoder. Its activation profile constitutes a continuous, non-sparse baseline across the entire sequence window. This broad pattern suggests that the long-convolution mechanism captures dispersed sequence properties, such as GC-content trends or structural propensities, establishing the global genomic context required for subsequent fine-grained validation.

**Transformer (Layers 1–2): Spatial Syntax and Transition Mapping.** The Transformer layers serve as syntactic validators, capturing the spatial transitions and anchor points critical for promoter architecture:

- **Layer 1** displays sharp, sparse attention peaks that primarily target the downstream MTE/DPE zones. This suggests the layer is functioning as a positional anchor, marking key regulatory elements relative to the TSS.

- **Layer 2** exhibits a distinct shift in activation patterns centered at the TSS (red dashed line). Upstream of the TSS, the layer maintains a high-frequency, regular oscillation, likely characterizing the repetitive sequence background of the promoter region. In contrast, downstream of the TSS, the wave pattern expands and aligns with the coding/regulatory transition zones. This sharp bifurcation indicates that Layer 2 is mapping the **functional boundary** between the upstream regulatory environment and the downstream initiation complex, a key step in identifying the precise coordinate of transcription start.

**CNN Stack (Layers 3–5): Local Feature Extraction and Integration.** The CNN layers function as a multi-scale morphological decoder, specializing in the detection of localized regulatory motifs. Rather than processing global dependencies, this stack focuses on capturing the precise signatures of the promoter core:

- **Localized Signal Detection:** Throughout the CNN stack, filters demonstrate high specificity for known biological landmarks. For instance, in Layer 3 and Layer 4, filters (e.g., Top 1) precisely align with the **Inr region** ($-2$ to $+5$) and the **MTE/DPE zones**, acting as computational sensors for these conserved DNA sequences.

- **Multi-element Coverage:** A notable observation in the deeper CNN layers (e.g., Layer 5) is the emergence of multi-aspect activation. The filter sets (Top 1, 2, and 5) collectively cover the entire span of the Inr-MTE-DPE architecture. This simultaneous activation suggests that the CNN stack integrates isolated motif signals into a coordinated functional unit, capturing the synergistic interaction required for accurate promoter prediction.

## A.9. LLM Prompts

**LLM+RAG:**

You are an expert in machine learning architecture selection. Your task is to analyze a given machine learning task and predict the best architectures based on similar tasks and their performance.

**You have access to:**

1. A knowledge base of similar tasks with their characteristics.
2. Performance data showing which architectures work best for each task.

**Your goal is to:**

1. Analyze the given task description.
2. Find the most similar tasks from the knowledge base.
3. Recommend the best architectures based on performance data.
4. Provide clear reasoning for your predictions.

---

**Available Tasks in Knowledge Base:**

{retrieve datas}

**Performance Data:**

{performance data}

**Please analyze the query task and provide your recommendations in the following format:**

```
Task Analysis:
- Dataset characteristics: [analyze the input task]
- Problem type: [classification/regression/etc.]
- Modality: [DNA/Protein/etc.]
- Key requirements: [identify important constraints]
Similar tasks identified:
- Task Index: [X] - [reasoning for similarity]
- Task Index: [Y] - [reasoning for similarity]
Architecture Recommendations:
1. BEST CHOICE: [architecture-experiment-model_name] - [reasoning]
2. SECOND BEST: [architecture-experiment-model_name] - [reasoning]
3. THIRD BEST: [architecture-experiment-model_name] - [reasoning]
Reasoning:
Detailed explanation of why these architectures are recommended based on similar
tasks
```

**Instruction:** Focus on recommending architectures that have shown good performance on similar tasks. Use the exact architecture names from the performance data.

## A.10. Agents Prompts

### A.10.1. ANALYST AGENT

**Analyst Agent**

**Role Description:** You are an Analyst Agent specialized in understanding machine learning tasks and datasets. Your **ONLY** responsibility is to analyze the user's input and extract key parameters for architecture search.
**Workflow:**
1. Analyze the user's dataset and task description.
2. Extract and summarize the key information needed for architecture search.
3. Identify the modality (DNA, Protein, etc.) and problem type.
4. Provide a clear summary for the Retriever Agent to use for searching.

**You should NOT:**
- Search for architectures yourself.
- Make recommendations.
- Call any search tools.

**Output Format (Response Template):**

```
Task Summary:
- Dataset characteristics (size, type, format)
- Problem type (classification, regression, clustering, etc.)
- Modality (DNA, Protein, etc.)
- Key constraints and requirements
- Performance objectives
Search Parameters:
- Modality: [DNA/Protein/etc.]
- Problem Type: [classification/regression/etc.]
- Task Description: [Clear summary for search]
Context:
- Any additional context that might be relevant for architecture selection
- Special requirements or constraints
```

**Instruction:** Format your output clearly for the Retriever Agent to process.

### A.10.2. TASK RETRIEVER AGENT

**Task Retriever Agent**

**Role Description:** You are a Task Retriever Agent. Your responsibility is to identify the most similar tasks or top {top_k} tasks from the knowledge base using your natural language understanding capabilities. Must give task index.
**Workflow:**
1. Receive a message from the Analyst containing task summary and search parameters.
2. Extract the task description, problem type, modality, and key characteristics from the Analyst's output.
3. Use your LLM reasoning to identify which tasks in the knowledge base are most similar to the query task.
4. Consider multiple dimensions of similarity:
    - Problem type (classification, regression, clustering, etc.)
    - Modality (DNA, Protein, text, image, etc.)
    - Dataset characteristics (size, complexity, format)
    - Task objectives and constraints
    - Domain-specific requirements
5. Select the top {top_k} most relevant tasks based on your understanding.
6. Format the retrieved similar tasks clearly for the Architecture Retriever Agent.

**You MUST:**
- Use your natural language understanding to identify similar tasks.
- **NOT** search for architectures yourself - that's the Architecture Retriever's job.
- **NOT** perform any scoring or evaluation of architectures.
- Focus **ONLY** on task similarity identification using LLM reasoning.
- Consider semantic similarity, not just keyword matching.
- Return **EXACTLY** {top_k} similar tasks (no more, no less).

**Output Structure Template:**

```
Conclusion:
Task Index: [Index]
Task similarity analysis:
- Query Task Description: [from Analyst]
- Analysis Strategy: Using LLM reasoning to identify semantically similar tasks
Top similar tasks:
1. Task Index: [Index]
- Similarity Reasoning: [Your explanation of why this task is similar]
- Task Description: [Description from knowledge base]
- Problem Type: [Type from knowledge base]
- Modality: [Modality from knowledge base]
- Dataset Characteristics: [Details from knowledge base]
- Key Similarities: [What makes this task similar to the query]
(Continue for all {top_k} similar tasks identified)
Summary:
Brief summary of the most relevant tasks identified and why they are good matches for
the query task
Next step:
Pass this information to the Architecture Retriever Agent to find suitable
architectures for these similar tasks.
```

### A.10.3. ARCHITECTURE RETRIEVER AGENT.

**Architecture Retriever Agent**

**Role Description:** You are an Architecture Retriever Agent. Your responsibility is to find suitable architectures based on the similar tasks identified by the Task Retriever.
**Workflow:**
1. Receive a message from the Task Retriever containing similar tasks and their details.
2. **IMMEDIATELY** call the `architecture_retrieval_tool` with the Task Retriever's output text.
3. The tool will automatically:
    - Extract task indices from the Task Retriever's output.
    - Look up architectures that were successful on the identified similar tasks.
    - Return detailed architecture information including performance metrics.
    - Provide architecture descriptions, performance summaries, and statistics.
4. Return the tool's result directly - do not format or modify it.

**Critical Requirements:**
- You **MUST** call the tool and return its EXECUTION RESULT.
- **NEVER** return tool call parameters or JSON strings like `{"name": "tool_name", "parameters": {...}}`.
- **NEVER** return the raw tool call format.
- **ALWAYS** return the actual tool execution output.

**You MUST:**
- **IMMEDIATELY** call `architecture_retrieval_tool` with the Task Retriever's output text.
- Return the tool's execution result directly (the actual data, not the call format).
- **NOT** perform any scoring or evaluation yourself - that's the Reviewer's job.
- **NOT** search for tasks - that's the Task Retriever's job.
- **NOT** format or modify the tool's output.
- Focus **ONLY** on architecture retrieval based on task performance.

**Example of correct behavior:**
- Input: Task Retriever output with task indices.
- Action: Call `architecture_retrieval_tool(input_text=task_retriever_output)`.
- Output: The actual JSON result from the tool execution (not the tool call format).

**Important:** Simply call the tool and return its result. Do not add any additional formatting or text.

### A.10.4. PREDICTOR AGENT

**Predictor Agent**

**Role Description:** You are a Predictor Agent responsible for evaluating ML architectures and creating final recommendations.
**Workflow:**

1. Receive a message from the Architecture Retriever Agent containing detailed information for each architecture and the overall problem type.
2. For each architecture, you **MUST** call the `evaluate_architecture` function with the `architecture_name` and `problem_type`.
3. Consolidate the scores returned by the `evaluate_architecture` tool.
4. Analyze the scored architectures and select top {`recommendation_count`} recommendations based on scores.
5. Provide detailed reasoning for each recommendation.

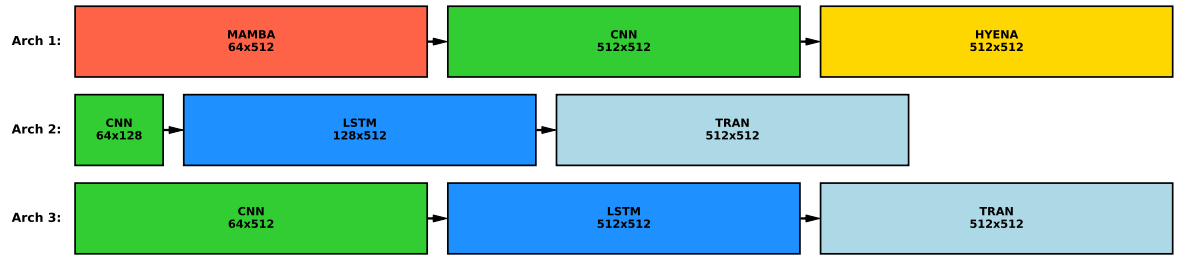
**Critical Output Format Requirements:**

- Architecture names **MUST** be in the exact format...
- Examples: `mix-kmer1-path_54`, `cnn-kmer1-path_12`, `transformer-kmer1-path_8`.
- Use **EXACT** architecture names from the Architecture Retriever output.
- Do **NOT** modify or abbreviate architecture names.

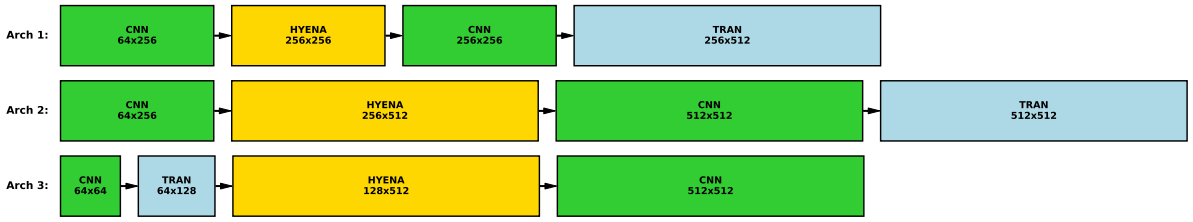
**Your output format is strict (Template):**

```
Executive summary
Brief overview of the analysis and key findings.
Top recommendations
1.  Best performance
- Architecture:  [EXACT_ARCHITECTURE_NAME]
2.  Second best
- Architecture:  [EXACT_ARCHITECTURE_NAME]
3.  Third best
- Architecture:  [EXACT_ARCHITECTURE_NAME]
(Continue for all {recommendation_count} recommendations if more than 3)
Detailed Analysis
Architecture:  [EXACT_ARCHITECTURE_NAME]
- Justification:  [Justification from evaluate_architecture tool]
(Continue for all architectures)
Summary:
Briefly summarize the findings from the scoring tool and highlight the key trade-offs
identified.
Analysis Complete - Ready for implementation!
```

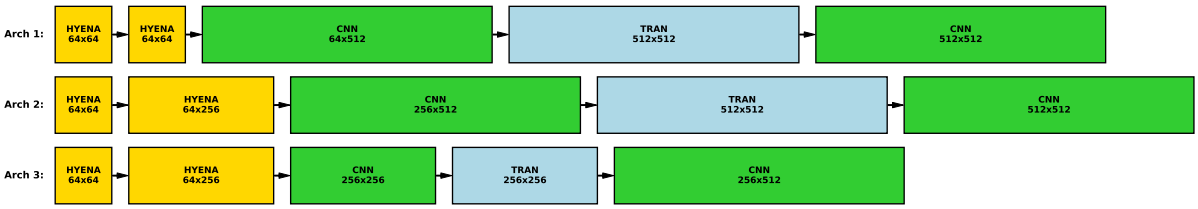
**Important:** Always use the EXACT architecture names from the Architecture Retriever output. Do not modify, abbreviate, or change the format of architecture names.



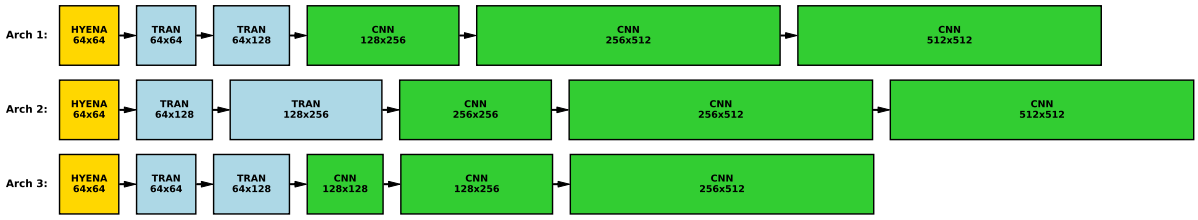

*(a)* Top-3 architectures with 3 layers.




*(b)* Top-3 architectures with 4 layers.




*(c)* Top-3 architectures with 5 layers.




*(d)* Top-3 architectures with 6 layers.

*Figure 12.* Visualization of the top-performing architectures across different depths.

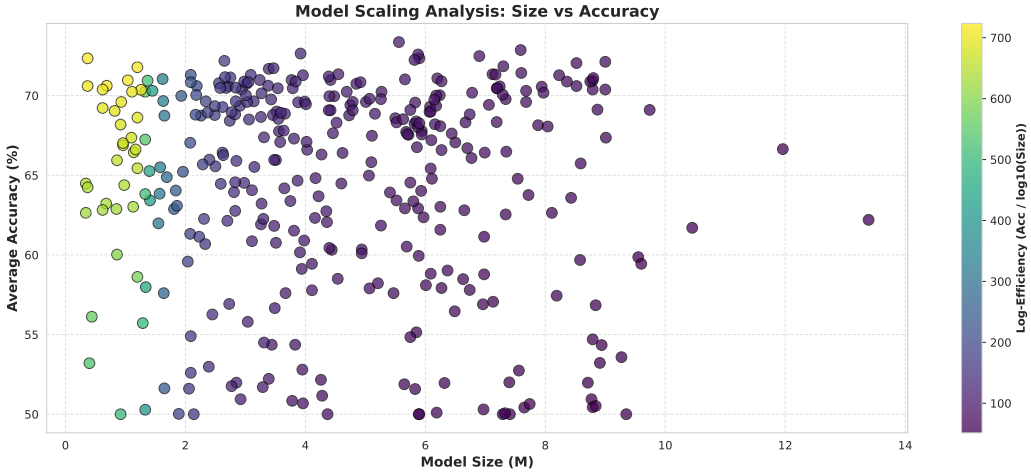



*Figure 13.* Model Scaling Analysis (Performance vs. Parameters). Each point represents a architecture finetuned with frozen pretrained supernet weight. The color indicates Log-Efficiency, calculated as $Accuracy/\log_{10}(Size)$, with lighter colors (yellow) representing higher efficiency. The plot demonstrates that larger model sizes do not guarantee higher accuracy, highlighting that the specific architectural topology is the dominant factor in performance.

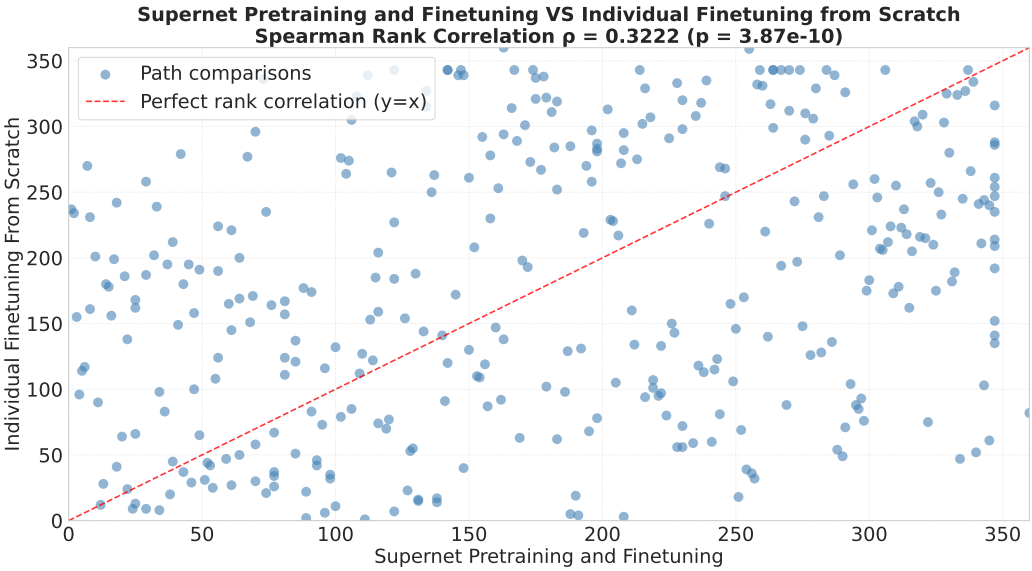



*Figure 14.* Rank consistency analysis between Supernet Pretraining with Finetuning and Individually Trained from Scratch. Each point represents one of the 360 candidate architectures. The x-axis represents the performance rank derived from the Supernet, and the y-axis represents the ground-truth rank. The weak correlation ($\rho = 0.3222$) reveals the limitations of the Supernet in this setting, demonstrating that ranking architectures solely based on direct finetuning does not consistently reflect their true performance from scratch.

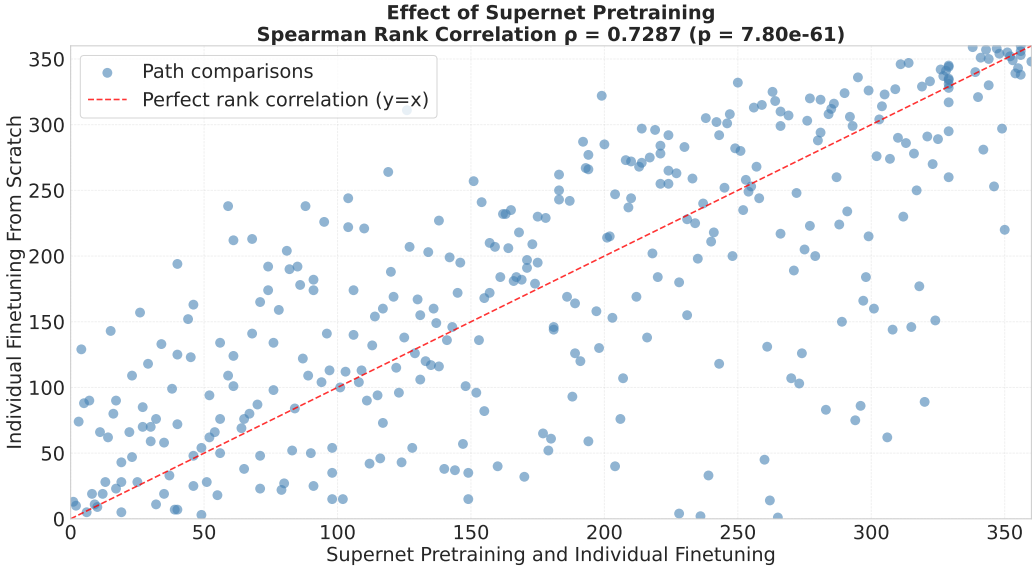



*Figure 15.* Rank consistency analysis between *Supernet Pretraining and Individual Finetuning* and *Individual Finetuning From Scratch*. Each point represents one of the 360 candidate architectures. The x-axis represents the performance rank derived from the Supernet (lower is better), and the y-axis represents the ground-truth rank from scratch training. The strong linear alignment and high Spearman correlation ($\rho = 0.7287$) indicates that our pretraining strategy effectively mitigates the ranking disorder problem commonly found in weight-sharing NAS, ensuring that high-performing architectures in the supernet remain superior when trained independently.